IT3010

Network Design and Management

Lecture 04
Introduction to SNMP

Shashika Lokuliyana

Faculty of Computing

Department of CSE

SLIIT

Discover Your Future

SNMP

Simple Network Management Protocol

Network management

- In computer networks, network management refers to the activities, methods, procedures, and tools that pertain to the operation, administration, maintenance, and provisioning of networked systems. Network management is essential to command and control practices and is generally carried out of a network operations center.

network management

1. *Operation* deals with keeping the network (and the services that the network provides) up and running smoothly.
2. *Administration* deals with keeping track of resources in the network and how they are assigned.
3. *Maintenance* is concerned with performing repairs and upgrades.
4. *Provisioning* is concerned with configuring resources in the network to support a given service.

network management

- Data for network management is collected through several mechanisms.
  - *Agents* installed on infrastructure
  - Synthetic monitoring that simulates transactions
  - Logs of activity
  - Sniffers
  - Real user monitoring

In the past network management mainly consisted of monitoring whether devices were up or down; today performance management has become a crucial part of the IT team's role which brings about a host of challenges—especially for global organizations.

Need For Network Management Tools

- In the early days of the Arpanet, the predecessor of the Internet, the name service was accomplished by maintaining and distributing one file with all the IP addresses of the network. But no more … DNS etc
- As networks increase in size

1. The network becomes more indispensable to the organization.
2. More things can go wrong, disabling or degrading the performance of portions of the network.

- Today a large network cannot be managed with software assistance.

SNMP & Network Management History

- ***1983*** - TCP/IP replaces ARPANET at U.S. Dept. of Defense, effective birth of Internet
- First model for net management - **HEMS** - High-Level Entity Management System (*RFCs 1021,1022,1024,1076*)
- ***1987*** - ISO OSI proposes **CMIP** - Common Management Information Protocol, and **CMOT** (CMIP over TCP) for the actual network management protocol for use on the internet
- ***Nov. 1987*** - **SGMP** - Simple Gateway Monitoring protocol (*RFC 1028*)
- ***1989*** - Marshall T. Rose heads up **SNMP** working group to create a common network management framework to be used by both **SGMP** and **CMOT** to allow for transition to **CMOT**

SNMP & Network Management History

- *Aug. 1989* - "**Internet-standard Network Management Framework**" defined (*RFCs 1065, 1066, 1067*)
- *Apr. 1989* - **SNMP** promoted to ***recommended*** status as the de facto TCP/IP network management framework (*RFC 1098*)
- *June 1989* - IAB committee decides to let **SNMP** and **CMOT** develop separately
- *May 1990* - IAB promotes **SNMP** to a **standard protocol with a recommended status** (*RFC 1157*)
- *Mar. 1991* - format of MIBs and traps defined (*RFCs 1212, 1215*)
- TCP/IP MIB definition revised to create **SNMPv1** (RFC 1213)

Technologies

- Main reason for having a standardized protocol was that without one in a heterogeneous environment, the network engineer would be spending too much time developing customized management tools instead of managing the network.
- A small number of accessory methods exist to support network and network device management. Access methods include,

- SNMP
- command-line interface
- custom XML
- CMIP
- Windows Management Instrumentation (WMI)
- Transaction Language 1
- CORBA
- NETCONF
- Java Management Extensions (JMX)

Use of Standardized Protocol

- Goals:
- Minimize complexity of management functions.
- Flexible, extensible.
- Independent of the architecture and mechanisms of particular hosts and gateways.

SNMP –
Simple Network Management Protocol

- "Internet-standard protocol for managing devices on IP networks"

- SNMP is a tool (protocol) that allows for remote and local management of items on the network including servers, workstations, routers, switches and other managed devices. It consists of a set of standards for network management, including an application layer protocol, a database schema, and a set of data objects.

Key components

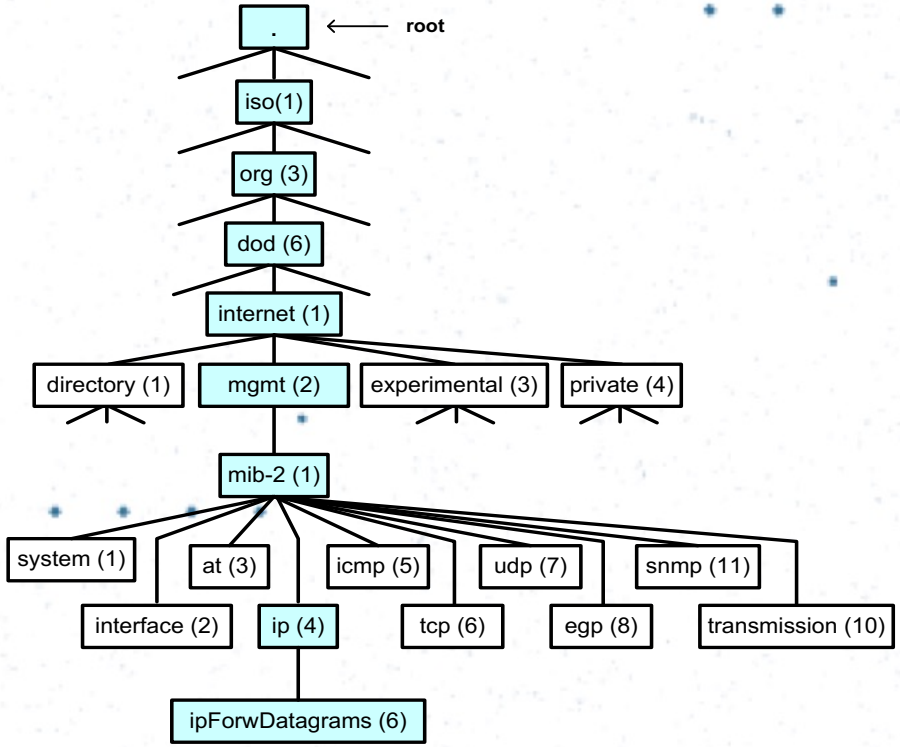
- **Manager** — an application program that contacts an SNMP agent to query or modify the database at the agent.
- **Agent** — software that runs on a piece of network equipment (host, router, printer, or others) and that maintains information about its configuration and current state in a database.
- **Network management station (NMS)** — executes applications that monitor and control managed devices. NMSs provide the bulk of the processing and memory resources required for network management. One or more NMSs may exist on any managed network.
- **Management Information Bases (MIBs)** — describes the information in the database.
- **SNMP Protocol** — application layer protocol used by SNMP agents and managers to send and receive data.

Network management station (NMS)

- The one that executes network management applications (NMAs) that monitor and control network elements (NE) such as hosts, gateways and terminal servers.
- These network elements use a management agent (MA) to perform the network management functions requested by the network management stations.

Management information base (MIB)

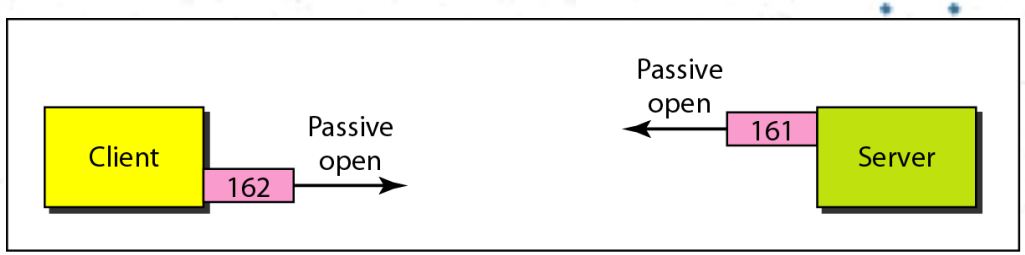
- MIBs describe the structure of the management data of a device subsystem; they use a hierarchical namespace containing object identifiers (OID).
- Each OID identifies a variable that can be read or set via SNMP.
- MIBs use the notation defined by Structure of Management Information Version 2 (SMIv2, RFC 2578), a subset of ASN.1(Abstract Syntax Notation One).



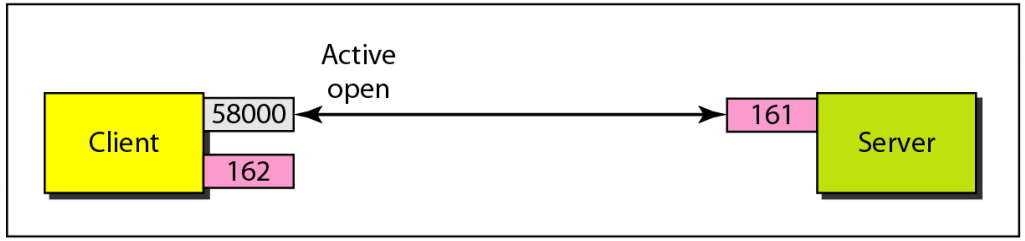
SNMP Protocol

- **SNMP versions**
  - **SNMPv1**
  - **SNMPv2**
  - **SNMPv3**

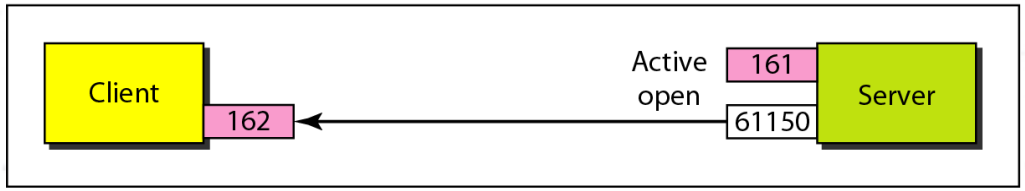
- Operates in the Application Layer.
- The SNMP Manager sends requests on UDP port 161.
- The SNMP Agent sends notifications on UDP port 162.
- SNMPv1 specifies five core protocol data units (PDUs). Two other PDUs were added in SNMPv2 and carried over to SNMPv3.



a. Passive open by both client and server



b. Exchange of request and response messages



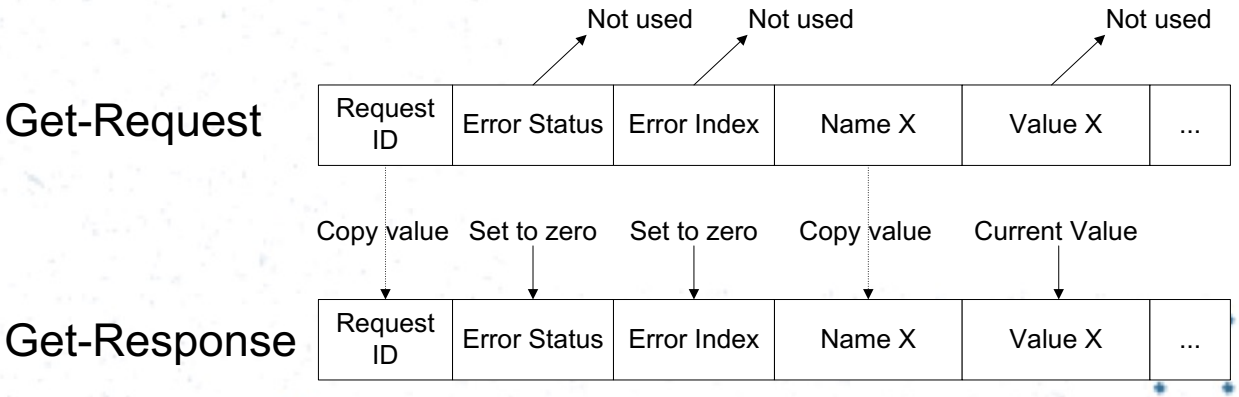
c. Server sends trap message

SNMP PDU Format

Sending a Request

1. Appropriate PDU is constructed based on the VarBindList and the operation type.
2. The PDU, security information and agent identity are input to a security mechanism to apply authentication and encryption to the PDU (This is only conceptual in SNMPv1). The result from the security mechanism and the community string are used to generate an SNMP message.
3. PDU type –Get Request type
4. The message is then serialized and sent using a transport service (i.e. UDP) to the specified SNMP agent (serialization is done using the BER encoding scheme).

PDU Type used



- **GetRequest**
- A manager-to-agent request to retrieve the value of a variable or list of variables. Desired variables are specified in variable bindings (values are not used). Retrieval of the specified variable values is to be done as an atomic operation by the agent. A *Response* with current values is returned.

SNMPv1

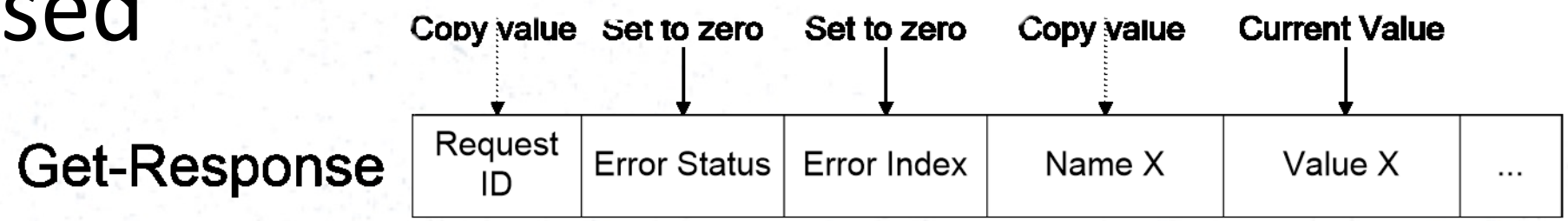
Receiving a Request

1. Incoming message is de-serialized to construct an ASN.1 message.
2. Version number is verified.
3. The community name, security information and the data found in the SNMP message are input to a security mechanism.
4. The agent then performs a rudimentary parse of the ASN.1 object returned from the security service to build an ASN.1 object corresponding to an SNMP PDU object.

Sending a Response

1. It first constructs a Response PDU using as input, the saved request-id value and the values for error-status, error-index and VarBindList returned from processing the request.
2. The PDU, security information and agent identity are input to a security mechanism. The result from the security mechanism and the community string are used to generate an SNMP message.
3. The message is serialized and then sent to the manger address from the request.

Pdu type used



- **Response**
- Returns variable bindings and acknowledgement from agent to manager for *GetRequest* or any other request message type. Error reporting is provided by error-status and error-index fields. Although it was used as a response to both gets and sets, this PDU was called *GetResponse* in SNMPv1.

SNMPv1

Receiving a Response

1. Incoming message is de-serialized.
2. PDU version is verified.
3. The community name, security information and the data found in the SNMP message are input in the security mechanism.
4. The ASN.1 object is parsed.

GetRequest/GetResponce Errors

- **tooBig** – the size of the response would exceed a local limitation (and the error-index field has no additional information)
- **noSuchName** – a specified instance of the management information is not available to be accessed – due to access control settings or instance does not exist
- **genErr** – specified instance of the management information is not available due to some other reason

Since only one error indication is returned in a get-response message and each identified instance of management information in the request can cause an error, a manager may have to try the GET operation many times before a get-response message is retimed with no error.

Other Request PDU types

- **GetNextRequest**
- A manager-to-agent request to discover available variables and their values. Returns a Response with variable binding for the lexicographically next variable in the MIB. The entire MIB of an agent can be walked by iterative application of *GetNextRequest* starting at OID 0. Rows of a table can be read by specifying column OIDs in the variable bindings of the request.

SNMPv1

Other Request PDU types

- **GetBulkRequest**
- Optimized version of *GetNextRequest*. A manager-to-agent request for multiple iterations of *GetNextRequest*. Returns a Response with multiple variable bindings walked from the variable binding or bindings in the request. PDU specific non-repeaters and max-repetitions fields are used to control response behavior. *GetBulkRequest* was introduced in SNMPv2.

SNMPv2

Other Request PDU types

- **SetRequest**
- A manager-to-agent request to change the value of a variable or list of variables. Variable bindings are specified in the body of the request. Changes to all specified variables are to be made as an atomic operation by the agent. A Response with (current) new values for the variables is returned.

SNMPv1

SetRequest/GetResponce Errors

- **badValue** – the value provided for the field may not comply with the data types or standards defined
- **noSuchName** – a specified instance of the management information is not available to be accessed – due to access control settings or instance does not exist
- **genErr** – specified instance of the management information is not available due to some other reason

Since only one error indication is returned in a get-response message and each identified instance of management information in the request can cause an error, a manager may have to try the GET operation many times before a get-response message is retimed with no error.

Other PDU types

- **Trap**
- Asynchronous notification from agent to manager. SNMP traps enable an agent to notify the management station of significant events by way of an unsolicited SNMP message. Includes current *sysUpTime* value, an *OID identifying* the type of trap and optional variable bindings.
- No response is expected.

SNMPv1

TRAP types

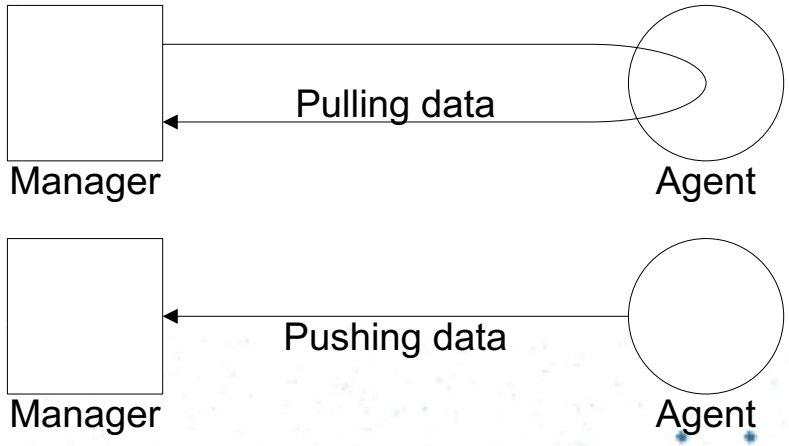
- **coldStart** - the sending protocol entity is reinitializing itself so that the agent's configuration or the protocol entity implementation can be altered.
- **warmStart** - the sending protocol entity is reinitializing itself so that neither the agent configuration nor the protocol entity implementation can be altered.
- **linkDown** - the sending protocol entity recognizes a failure in one of the communication links represented in the agent's configuration.
- **linkUp** - the sending protocol entity recognizes that one of the communication links represented in the agent's configuration has come up.

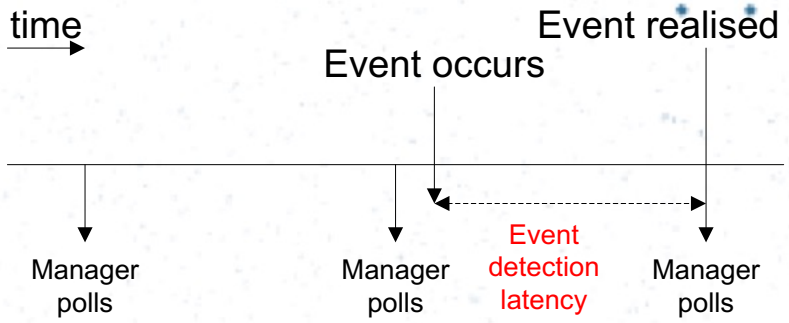
TRAP types

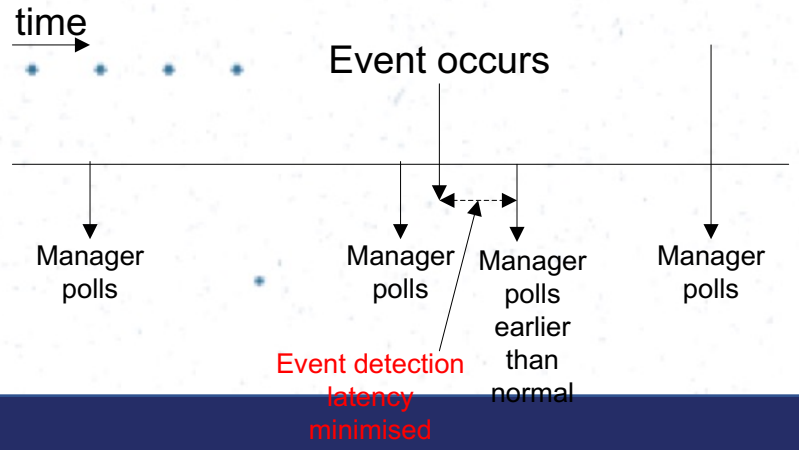
- **authenticationFailure** - the sending protocol entity is the addressee of a protocol message that is not properly authenticated.
- **egpNeighborLoss** - an EGP neighbor for whom the sending protocol entity was an EGP peer has been marked down and the peer relationship no longer exists.
- **enterpriseSpecific** - the sending protocol entity recognizes that some enterprise-specific event has occurred.

Use of Events

- Agents provide the specified data only when requested by managers.
  - This periodic gathering of data is called polling.
- Amount of time between an event and a manager realizing that an event has occurred is called the *event detection latency*.
- Another model of management is based on agents in managed systems sending data to configured managers.
  - Traps (Pushing).
  - However, during a communication failure, an event report would never reach a manager.







SNMP versions

1. **SNMPv1** - SNMP version 1 (SNMPv1) is the initial implementation of the SNMP protocol. Version 1 has been criticized for its poor security. Authentication of clients is performed only by a "*community string*".
2. **SNMPv2** - Revises version 1 and includes improvements in the areas of performance, security, confidentiality, and manager-to-manager communications. It introduced *GetBulkRequest*, an alternative to iterative *GetNextRequests* for retrieving large amounts of management data in a single request.
3. **SNMPv3** - SNMPv3 primarily added security and remote configuration enhancements to SNMP.

SNMPv1

SNMPv1 - Limitations

- Limited data types
  - Reduction in complexity of SNMP has resulted in some increase in complexity of MIB understanding and design.
- Limited performance
  - Inefficient for large retrievals. (e.g. retrieving entire MIB or tables)
    - E.g. 2000 entry table with 4 columns 200ms RTT Using GETNEXT Requests: 2000 X 4 X 2 = 16000 packets or 2000 X 4 X .2 = 1600 seconds…!!!
- Limited error codes
- Lack of hierarchies
  - Inherently centralized
- Lack of security
  - Community strings

SNMPv2

SNMPv2 - features

❖ New data types

	SMIv1	SMIv2
SIMPLE TYPES:	INTEGER OCTET STRING OBJECT IDENTIFIER	INTEGER OCTET STRING OBJECT IDENTIFIER
	-	Integer32
APPLICATION-WIDE TYPES:	- Gauge Counter - TimeTicks IpAddress Opaque NetworkAddress	Unsigned32 Gauge32 Counter32 Counter64 TimeTicks IpAddress Opaque -
PSEUDO TYPES:	-	BITS

SNMPv2 - features

- Community based (similar to SNMPv1).
- *GET and GETNEXT* operations are the same as SNMPv1.
- SET Request operation is a conceptual two phase commit;
  - PHASE 1 : Perform various checks.
  - PHASE 2 : Perform the actual set operation.
- SNMPv2 adds and enhances some protocol operations.
  - The *INFORM* Request.
  - The *GETBULK* Request.

GETBULK Request

Some further insight…

- For efficient retrieval of many VarBinds.
- GETBULK Request has two additional parameters:
- **non-repeators**
  - The first N elements (non-repeators) of the VarBind list are treated as if the operation was a normal GETNEXT Request operation.
- **max-repetitions**
  - The next elements of the VarBind list are treated as if the operation consisted of a number (max-repetitions) of repeated GETNEXT Request operations.

GETBULK Request

Some further insight…

REQUEST(non-repeaters = N; max-repetitions = M;
VariableBinding-1; … ; VariableBinding-N; VariableBinding-(N+1); … ; VariableBinding-(N+R)
)

N-TIMES

RESPONSE(
VariableBinding-1; … ; VariableBinding-N; VariableBinding-(N+1); … ; VariableBinding-(N+R)
VariableBinding-(N+1); … ; VariableBinding-(N+R)
VariableBinding-(N+1); … ; VariableBinding-(N+R)
…
VariableBinding-(N+1); … ; VariableBinding-(N+R)
)

M-TIMES

1st LEXICOGRAPHICAL SUCCESSOR

2nd LEXICOGRAPHICAL SUCCESSOR

3th LEXICOGRAPHICAL SUCCESSOR

Mth LEXICOGRAPHICAL SUCCESSOR

INFORM Request

Some further insight…

- Remember that for SNMPv1 trap messages, there is no response sent by the manager. Hence, no guarantee that the manager received the trap or not…!!!
- As a solution for this, SNMPv2 INFORM operation was introduced.
- In simple terms this is a confirmed trap.
  - Originally to INFORM a higher level manager
  - Same format as Trap PDU
  - Possible error: tooBig

SNMPv2 - features

❖Additional error/exceptions

- GET additional errors:
  - noSuchObject.
  - noSuchInstance.
- GETNEXT additional errors:
  - endOfMibView.
- Exceptions are coded within the VarBinds and do not raise Error Status and Index.

SNMPv2 - Limitations

- Too complex
- Overshoot problem - GETBULK does not stop when it reaches the end of a table.
- Although SNMPv2 was originally intended to overcome the security problem of SNMPv1 because of the high complexity, without the controversial new SNMP v2 security model, using instead the simple community-based security scheme of SNMPv1

SNMPv3

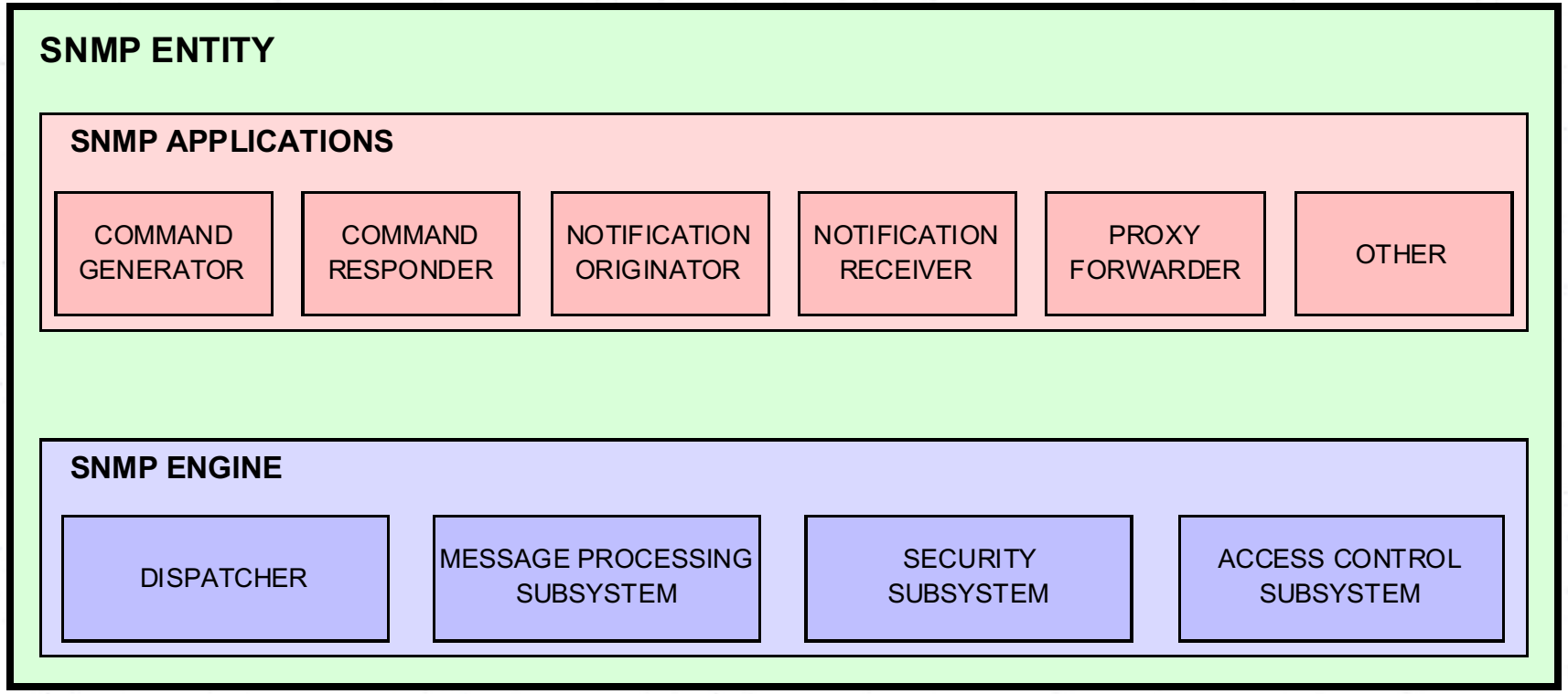
SNMPv3 - features

- Security.
- Remote configuration capabilities.
- Ability to control the agents via use of a newly defined specific management MIB.
- SNMPv3 contains all the functionality of SNMPv1 and SNMPv2.
- Incorporates the SNMPv2 data types.

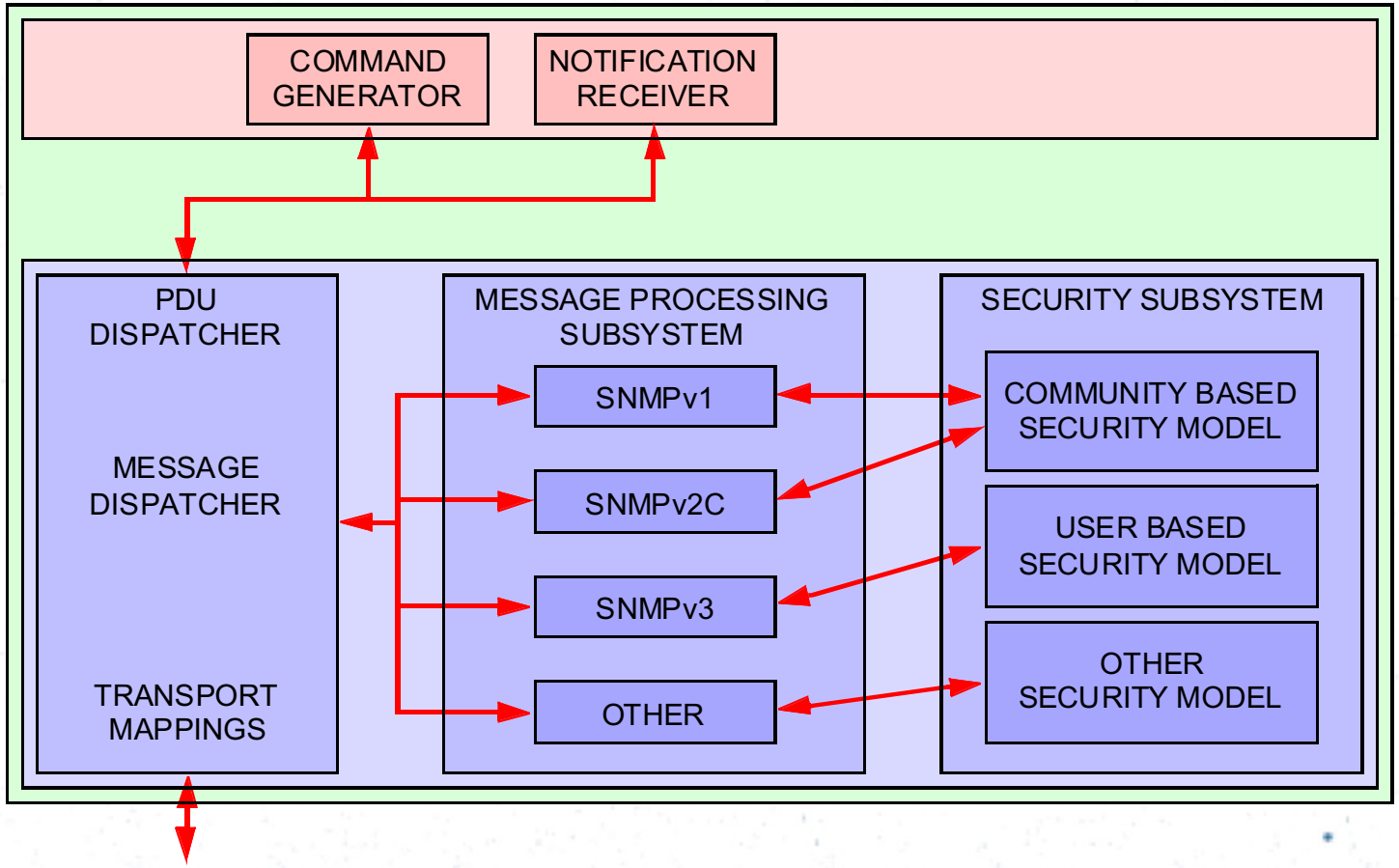
SNMPv3 - Entities

❖SNMP Manager and Agent is created as an SNMP Entity

- Entity consists of an **SNMP engine** and **SNMP applications**.



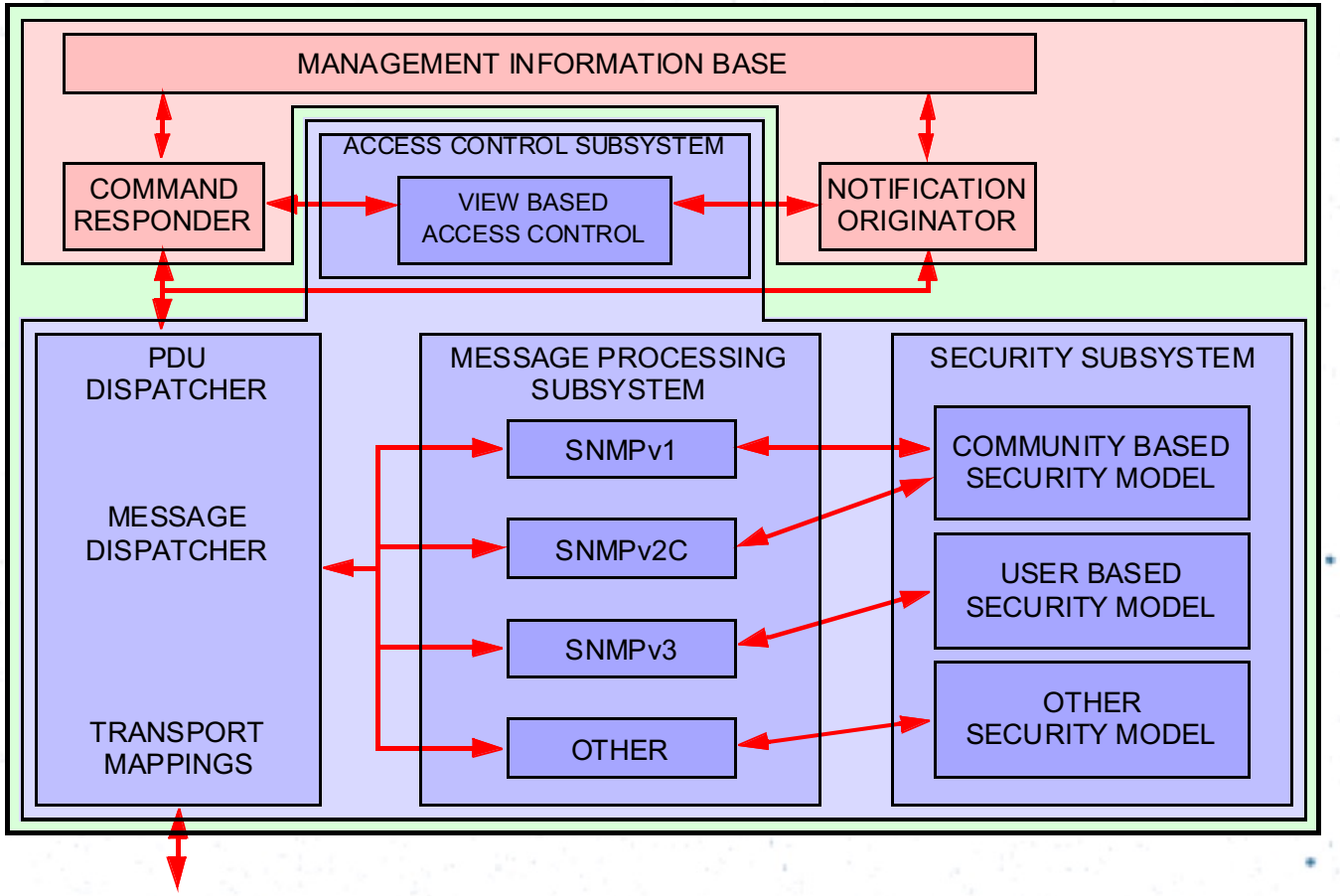
SNMPv3 – SNMP Manager entity



SNMP Application

SNMP Engine

SNMPv3 – SNMP AGENT entity



SNMP Application

SNMP Engine

SNMP Engine

dispatcher

- The dispatcher is a traffic manager that sends and receives messages.
- Incoming messages
  - The dispatcher tries to determine the version number of the message and then passes the message to the appropriate message processing model.
- Outgoing messages
  - The dispatcher selects the appropriate transports for sending messages. Currently this is UDP in the TCP/IP world.
- The dispatcher is also responsible for dispatching PDUs to applications.

SNMP Engine

Message processing subsystem

- Outgoing messages
  - Accepts outgoing PDUs from the dispatcher and prepares them for transmission by wrapping them in a message header.
- Incoming messages
  - Accepts incoming messages from the dispatcher, processes each message header.
- An implementation of the message processing subsystem may support a single message format corresponding to a single version of SNMP (SNMPv1, SNMPv2c, SNMPv3), or it may contain a number of modules, each supporting a different version of SNMP.

SNMP Engine

security subsystem

- Authenticates and encrypts messages.
- Outgoing messages are passed to the security subsystem from the message processing subsystem.
  - Depending on the services required, the security subsystem may encrypt the enclosed PDU and some fields in the message header.
  - The security subsystem may generate an authentication code and insert it into the message header.
- After encryption, the message is returned to the message processing subsystem.

SNMP Engine

security subsystem

- Incoming messages are passed to the security subsystem from the message processing subsystem.
  - If required, the security subsystem checks the authentication code and performs decryption.
  - The processed message is returned to the message processing subsystem.
  - An implementation of the security subsystem may support one or more distinct security models.

SNMP Engine

security subsystem

Consists of three sub modules,

1. Community Based Security Model
   - For SNMPv1 and SNMPv2
2. User Based Security Model
   - For SNMPv3 covering the CIA concepts
3. Other Security Model

SNMP Engine

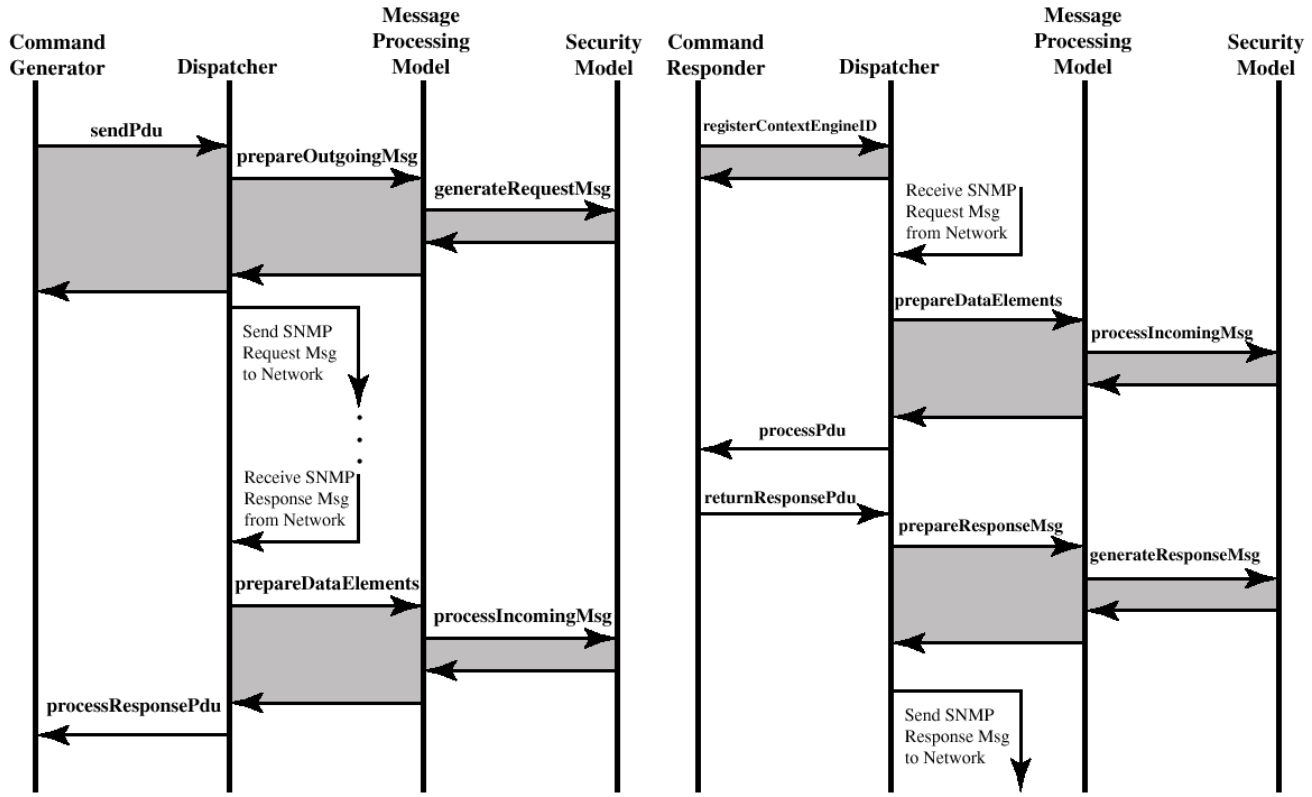
Access Control Subsystem

- Provides authentication.
  - Determine whether access to a managed object should be allowed.
- Can control which users and which operations can have access to which managed objects.

SNMP Application

- There are five types of applications.
  - **Command generators** - Generate SNMP commands to collect or set management data.
  - **Command responders** - Provide access to management data.
    - For example, processing get, get-next, get-bulk and set PDUs.
    - Commands are used in a command responder application.
  - **Notification originators** - Initiate Trap or Inform messages.
  - **Notification receivers** - Receive and process Trap or Inform messages.
  - **Proxy forwarders** - Forward messages between SNMP entities.

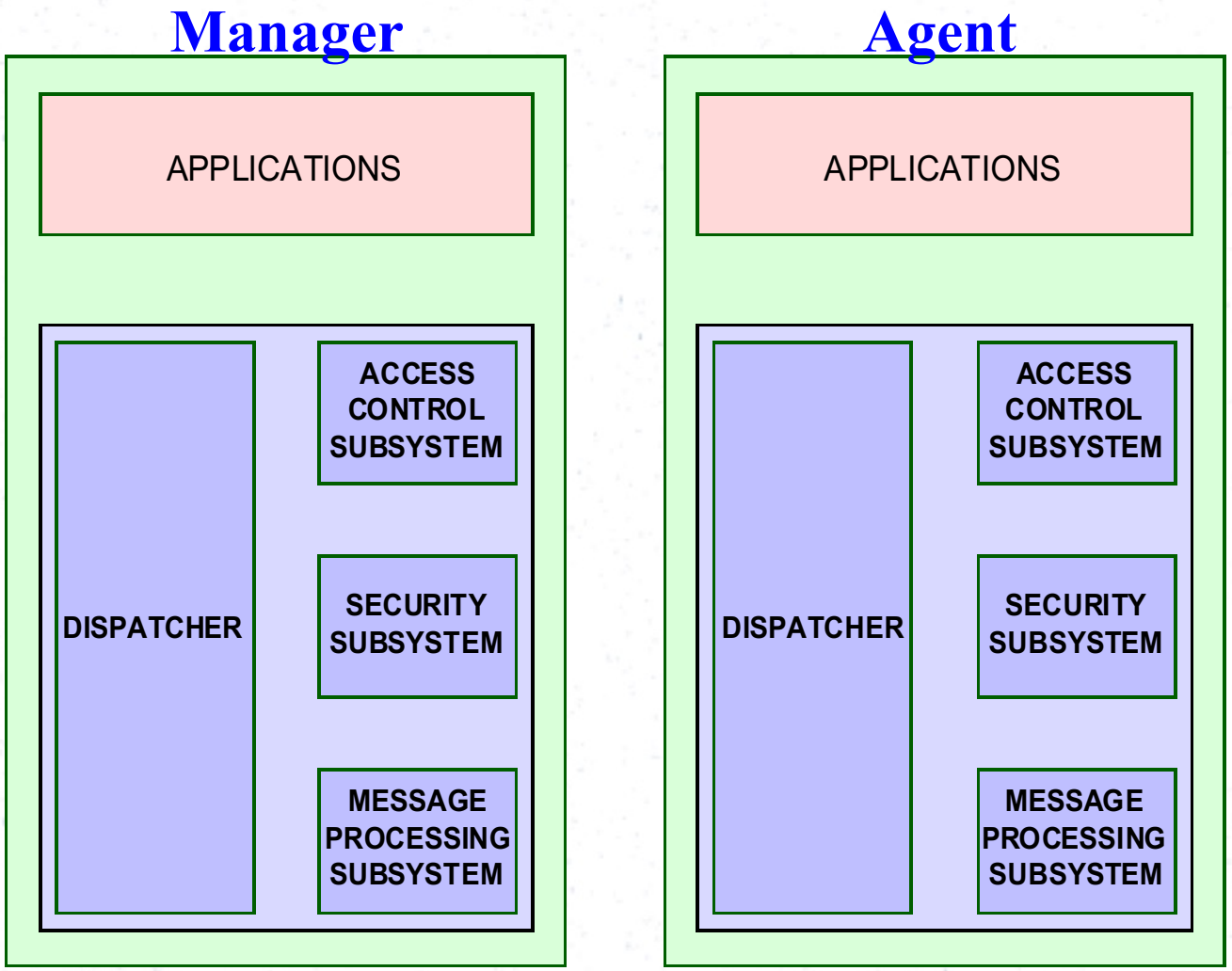
SNMPv3 flow



(a) Command Generator or Notification Originator

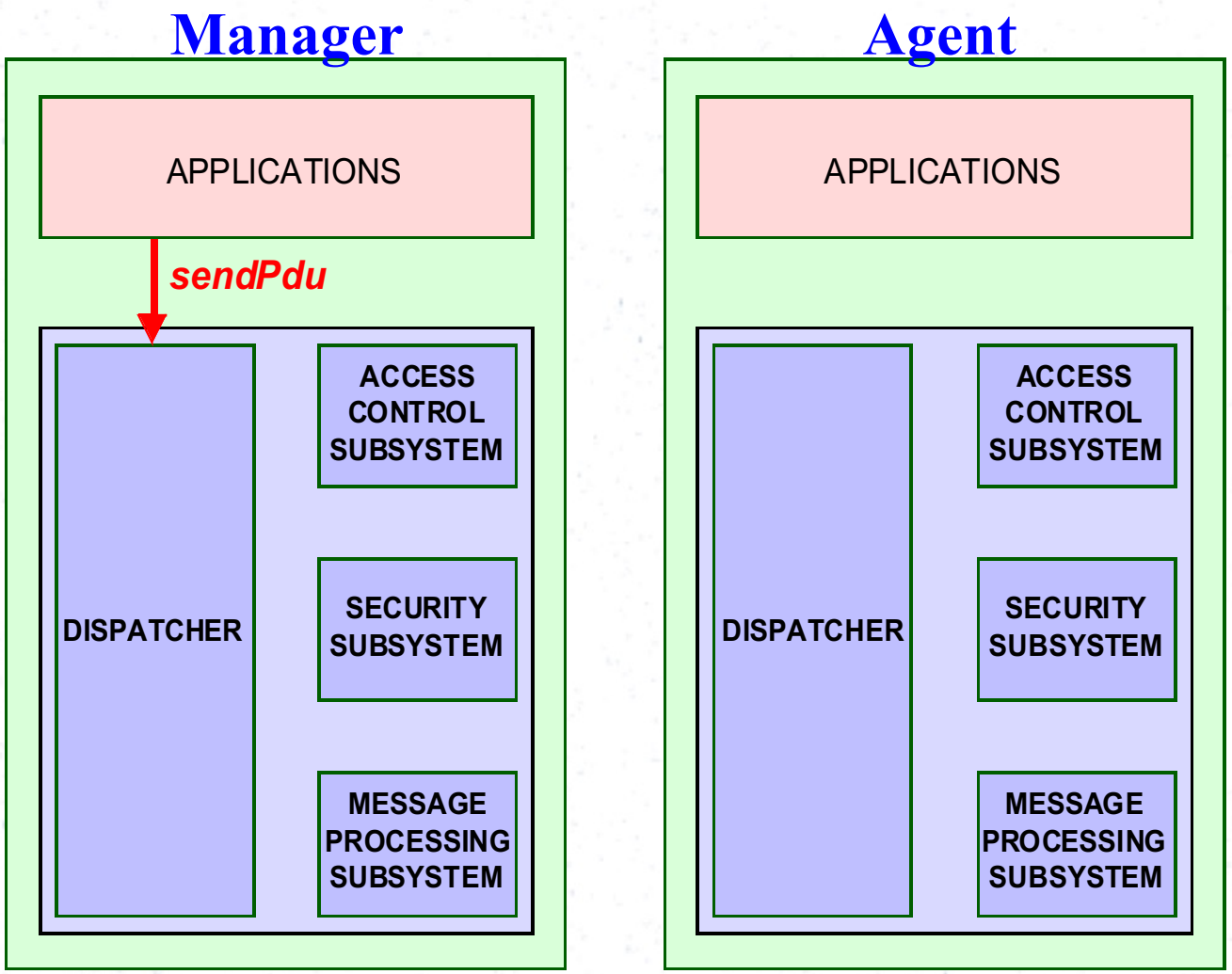
(b) Command Responder

Primitives Between Modules



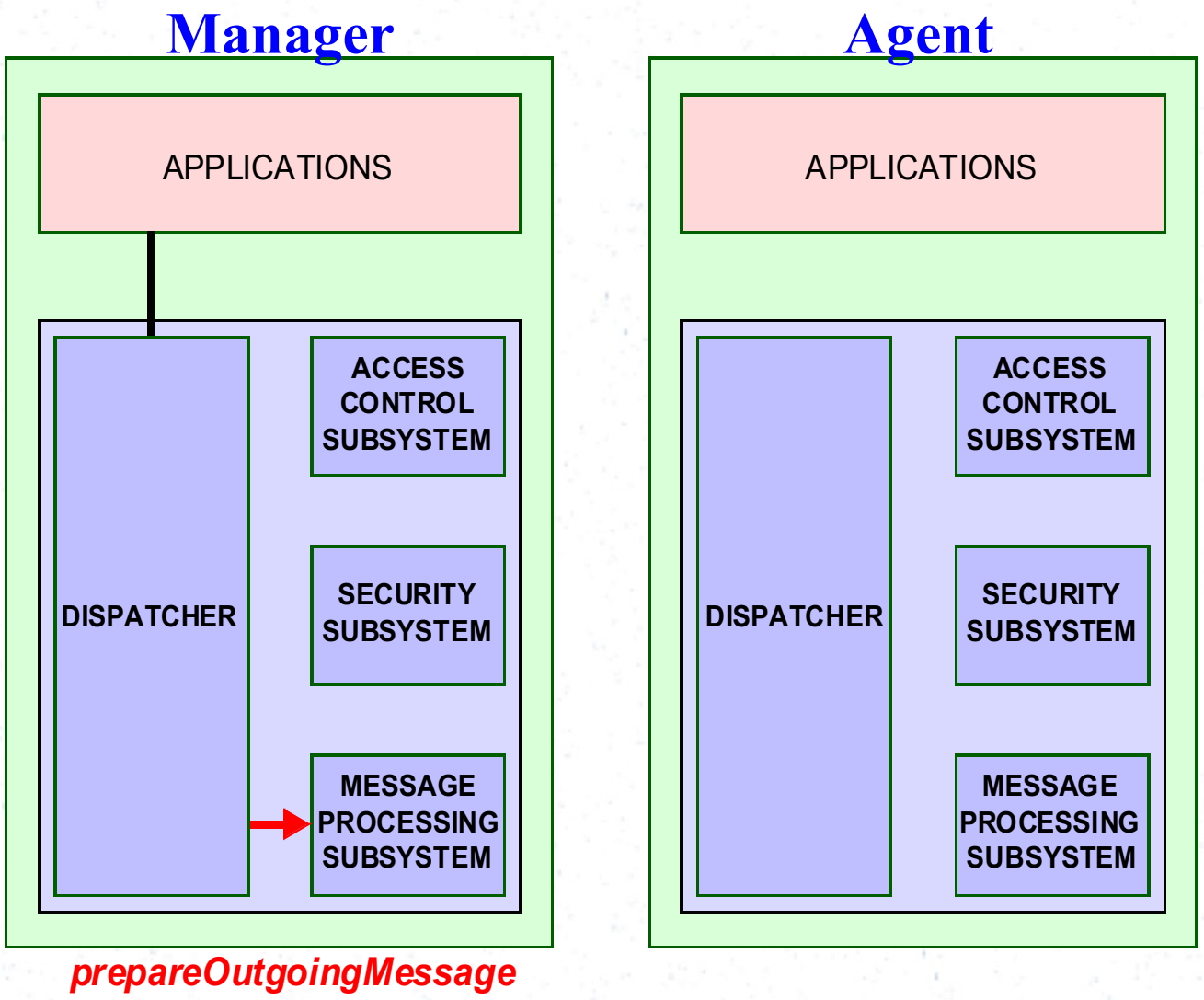
Parameters
contextEngineID
contextName
destTransportAddress
destTransportDomain
expectResponse
globalData
maxMessageSize
maxSizeResponseScopedPDU
messageProcessingModel
outgoingMessage
outgoingMessageLength
PDU
pduType
pduVersion
scopedPDU
stateReference
statusInformation
securityEngineID
securityLevel
securityModel
securityName
securityParameters
securityStateReference
sendPduHandle
transportAddress
transportDomain
variableName
viewType
wholeMsg
wholeMsgLength

sendPdu



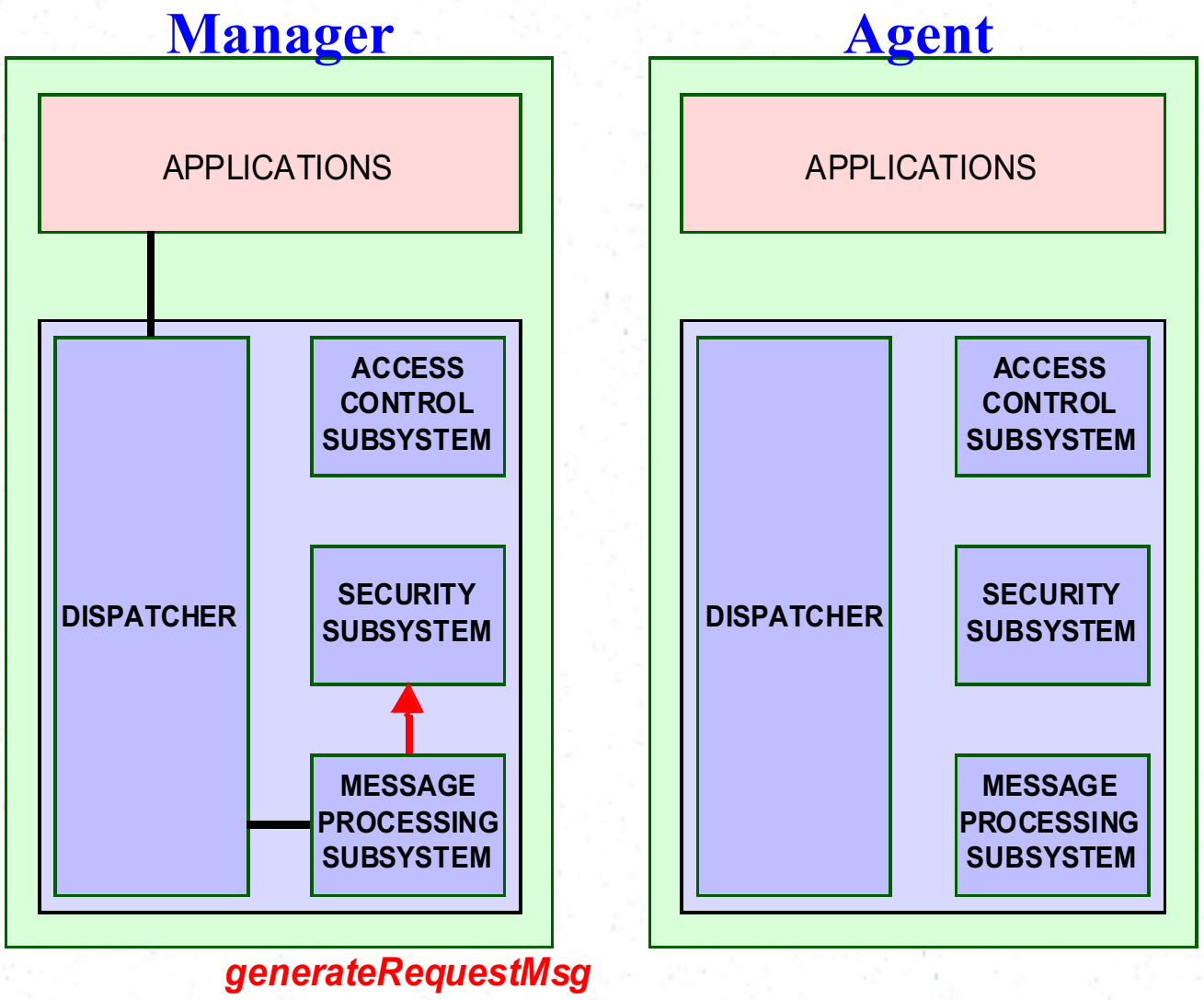
Parameters
contextEngineID
contextName
destTransportAddress
destTransportDomain
expectResponse
globalData
maxMessageSize
maxSizeResponseScopedPDU
messageProcessingModel
outgoingMessage
outgoingMessageLength
PDU
pduType
pduVersion
scopedPDU
stateReference
statusInformation
securityEngineID
securityLevel
securityModel
securityName
securityParameters
securityStateReference
sendPduHandle
transportAddress
transportDomain
variableName
viewType
wholeMsg
wholeMsgLength

prepareOutgoingMessage



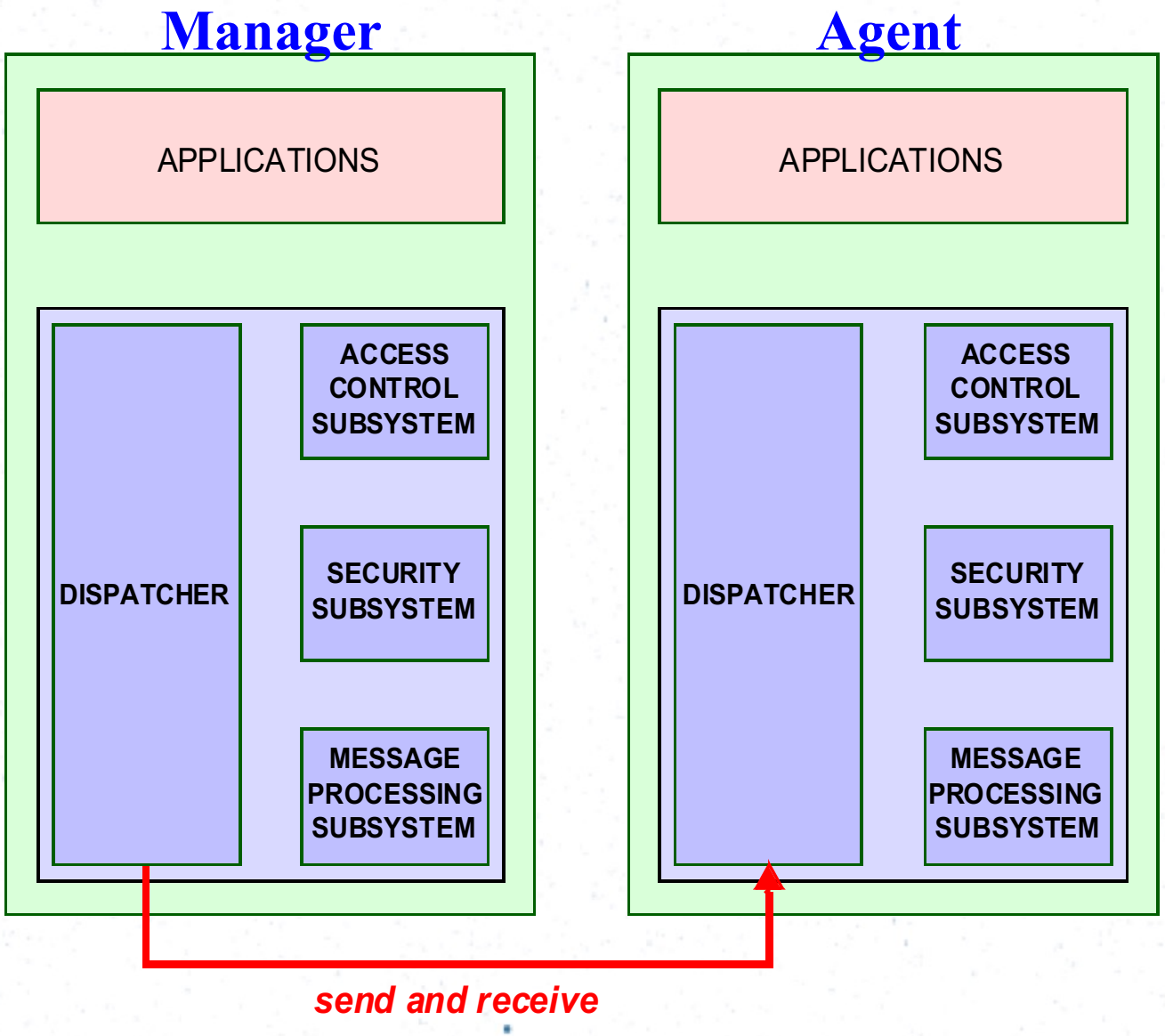
Parameters
contextEngineID
contextName
destTransportAddress
destTransportDomain
expectResponse
globalData
maxMessageSize
maxSizeResponseScopedPDU
messageProcessingModel
outgoingMessage
outgoingMessageLength
PDU
pduType
pduVersion
scopedPDU
stateReference
statusInformation
securityEngineID
securityLevel
securityModel
securityName
securityParameters
securityStateReference
sendPduHandle
transportAddress
transportDomain
variableName
viewType
wholeMsg
wholeMsgLength

generateRequestMsg



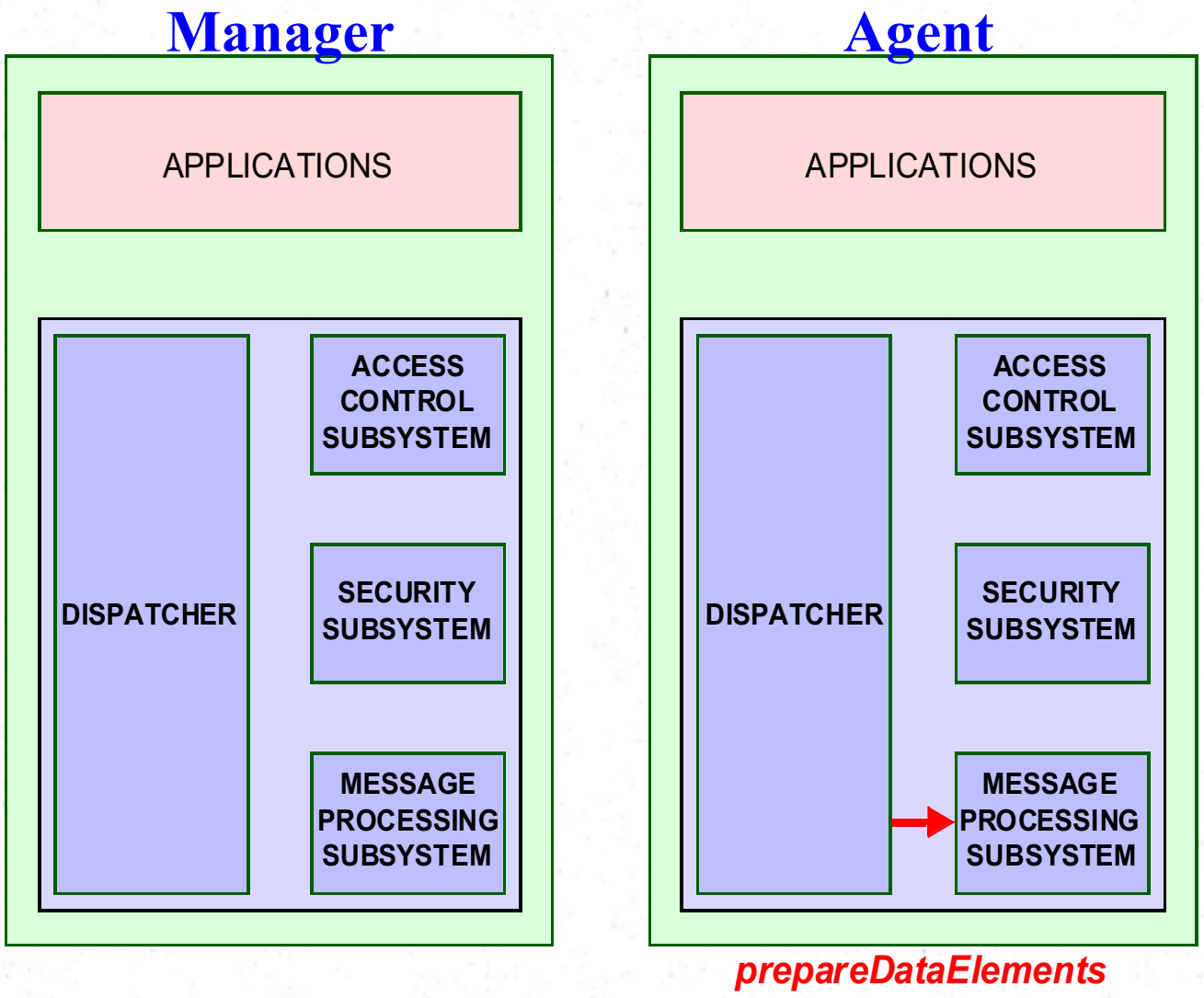
Parameters
contextEngineID
contextName
destTransportAddress
destTransportDomain
expectResponse
globalData
maxMessageSize
maxSizeResponseScopedPDU
messageProcessingModel
outgoingMessage
outgoingMessageLength
PDU
pduType
pduVersion
scopedPDU
stateReference
statusInformation
securityEngineID
securityLevel
securityModel
securityName
securityParameters
securityStateReference
sendPduHandle
transportAddress
transportDomain
variableName
viewType
wholeMsg
wholeMsgLength

send / receive



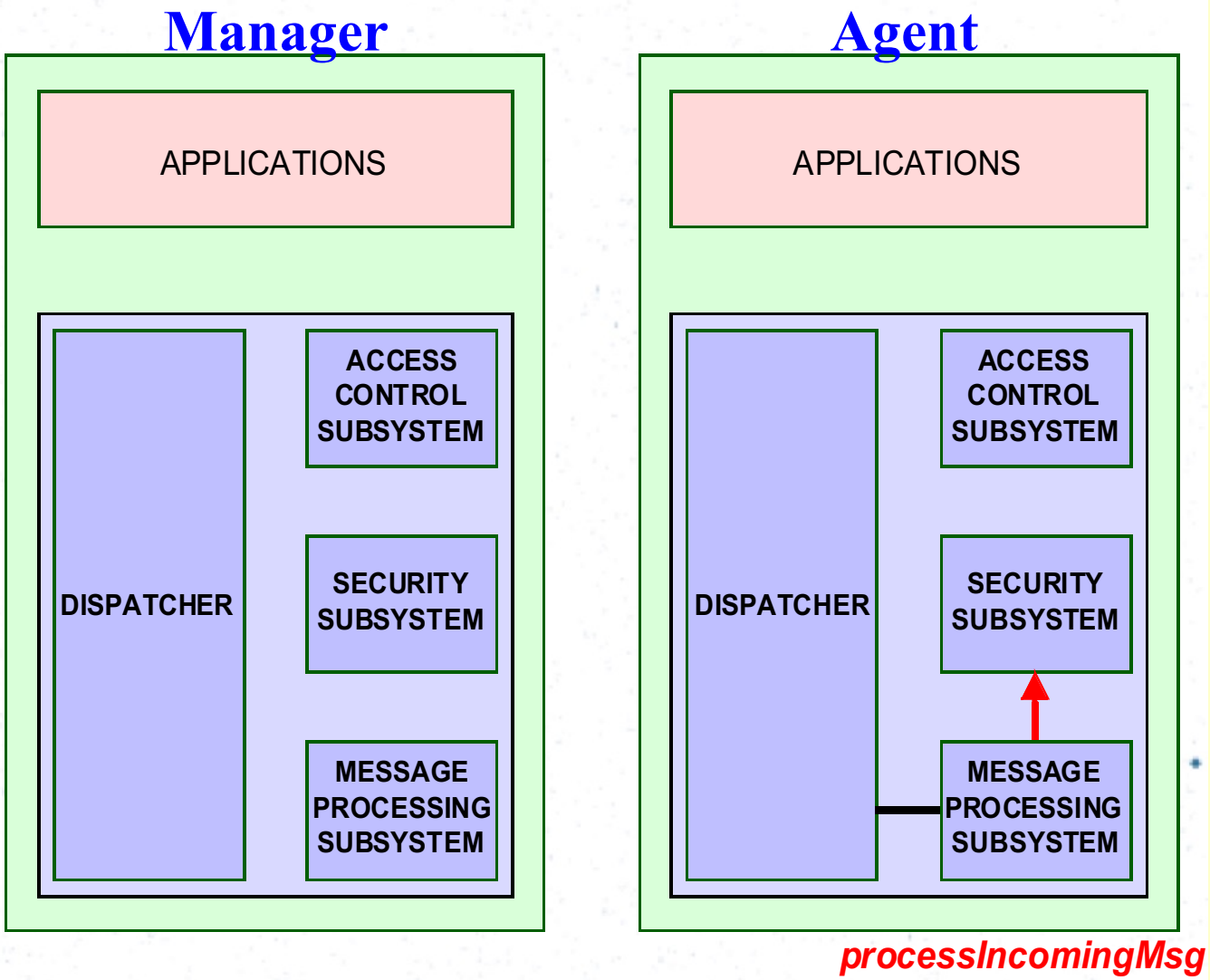
Parameters
contextEngineID
contextName
destTransportAddress
destTransportDomain
expectResponse
globalData
maxMessageSize
maxSizeResponseScopedPDU
messageProcessingModel
outgoingMessage
outgoingMessageLength
PDU
pduType
pduVersion
scopedPDU
stateReference
statusInformation
securityEngineID
securityLevel
securityModel
securityName
securityParameters
securityStateReference
sendPduHandle
transportAddress
transportDomain
variableName
viewType
wholeMsg
wholeMsgLength

prepareDataElements



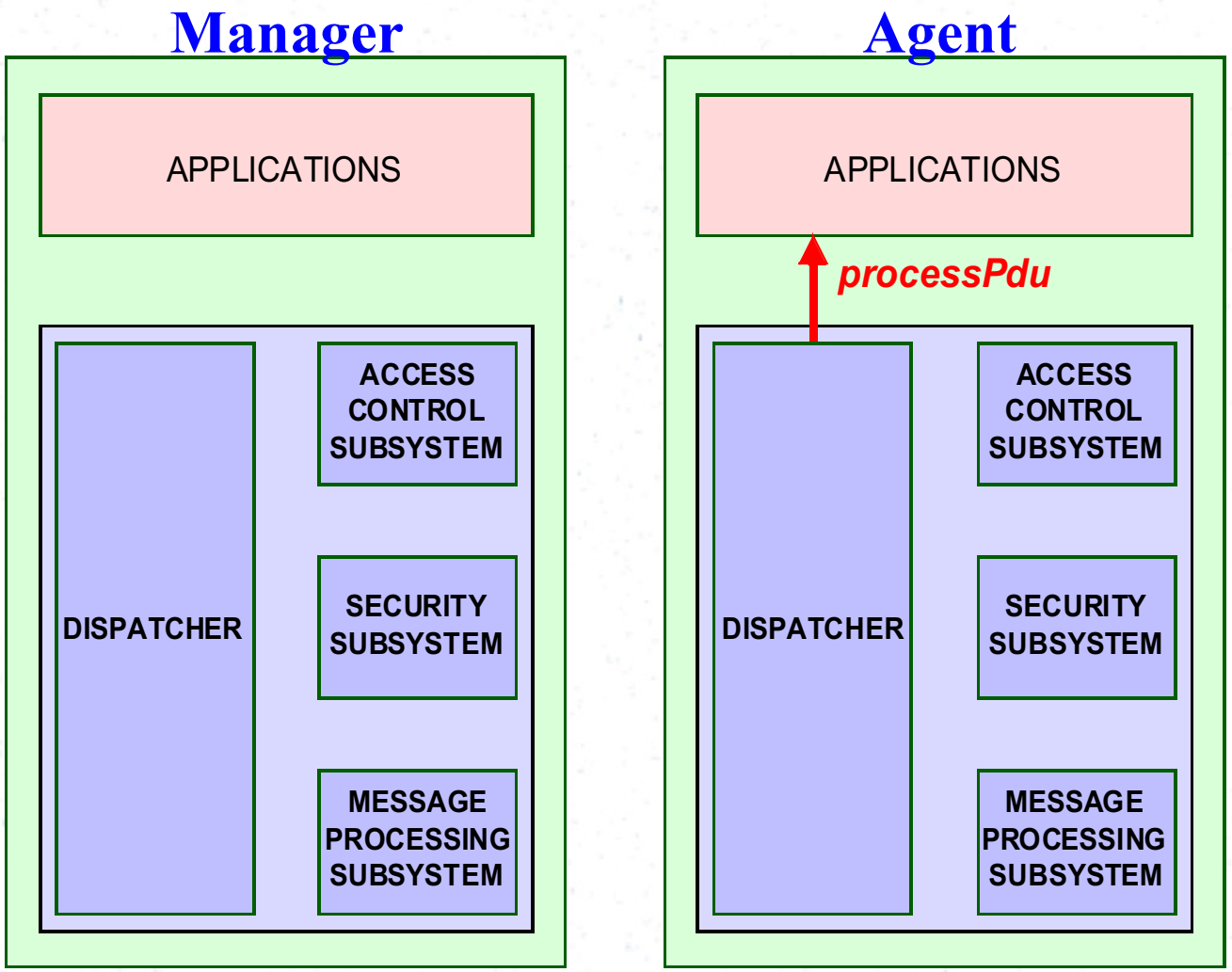
Parameters
contextEngineID
contextName
destTransportAddress
destTransportDomain
expectResponse
globalData
maxMessageSize
maxSizeResponseScopedPDU
messageProcessingModel
outgoingMessage
outgoingMessageLength
PDU
pduType
pduVersion
scopedPDU
stateReference
statusInformation
securityEngineID
securityLevel
securityModel
securityName
securityParameters
securityStateReference
sendPduHandle
transportAddress
transportDomain
variableName
viewType
wholeMsg
wholeMsgLength

processIncomingMsg



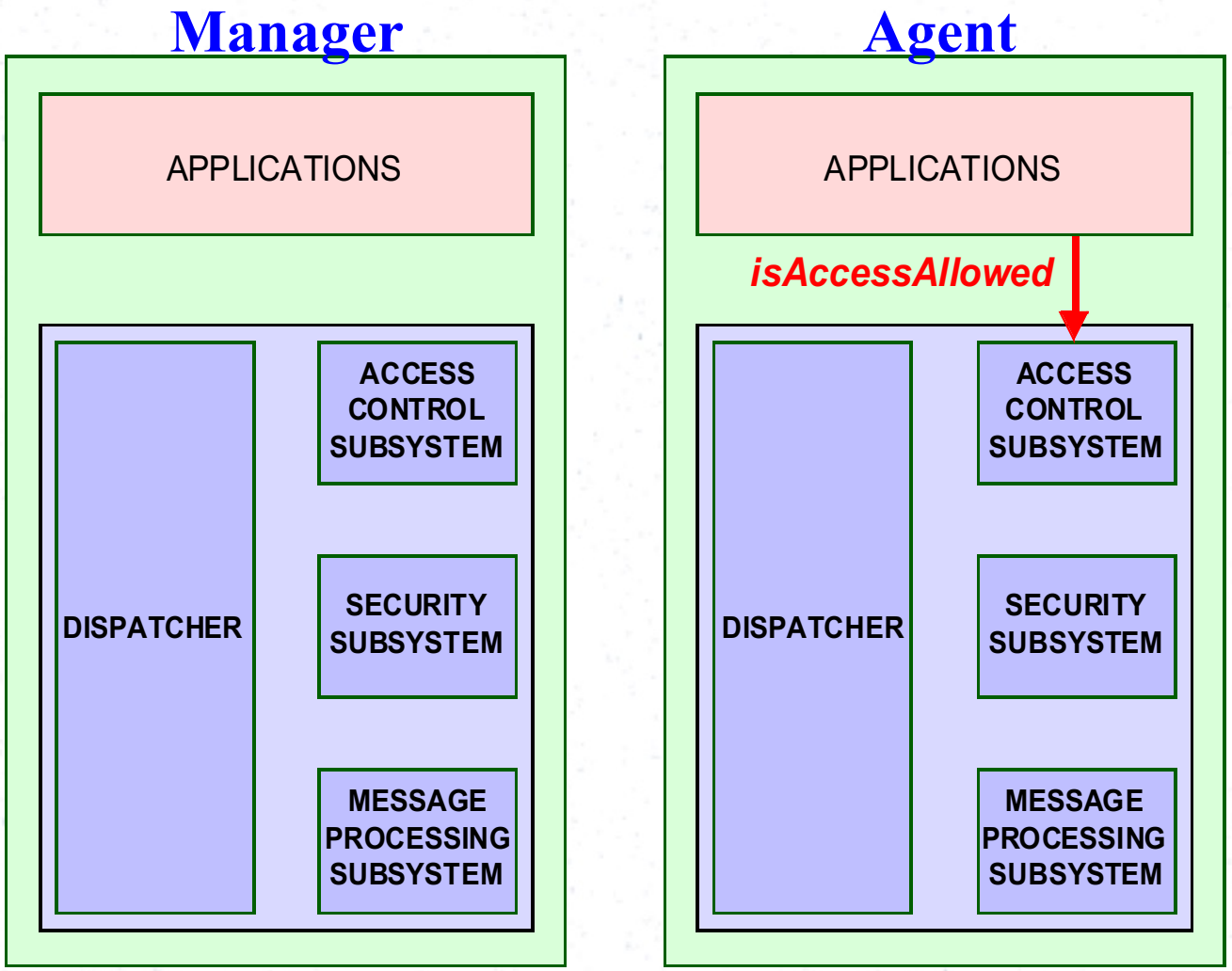
Parameters
contextEngineID
contextName
destTransportAddress
destTransportDomain
expectResponse
globalData
maxMessageSize
maxSizeResponseScopedPDU
messageProcessingModel
outgoingMessage
outgoingMessageLength
PDU
pduType
pduVersion
scopedPDU
stateReference
statusInformation
securityEngineID
securityLevel
securityModel
securityName
securityParameters
securityStateReference
sendPduHandle
transportAddress
transportDomain
variableName
viewType
wholeMsg
wholeMsgLength

processPdu



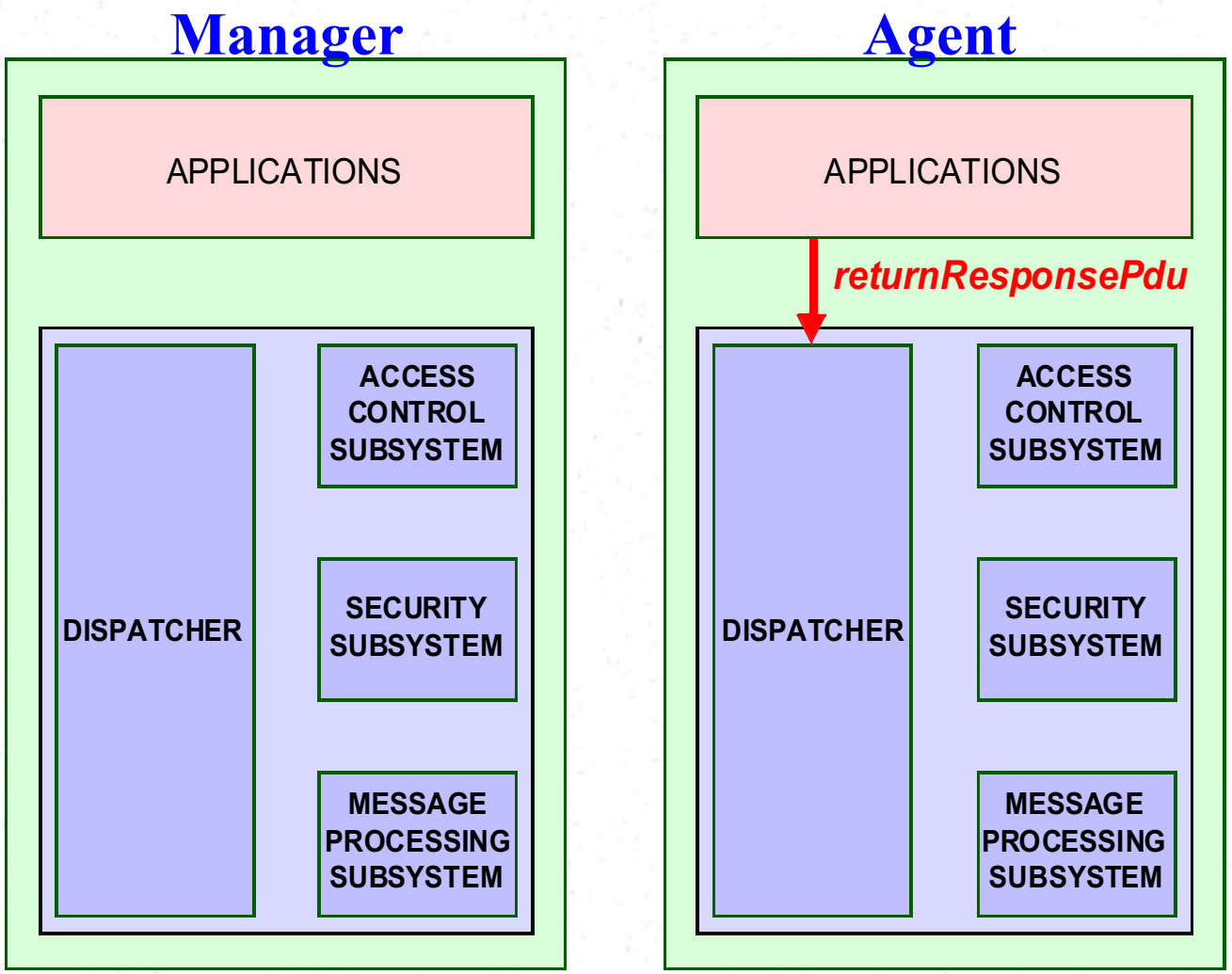
Parameters
contextEngineID
contextName
destTransportAddress
destTransportDomain
expectResponse
globalData
maxMessageSize
maxSizeResponseScopedPDU
messageProcessingModel
outgoingMessage
outgoingMessageLength
PDU
pduType
pduVersion
scopedPDU
stateReference
statusInformation
securityEngineID
securityLevel
securityModel
securityName
securityParameters
securityStateReference
sendPduHandle
transportAddress
transportDomain
variableName
viewType
wholeMsg
wholeMsgLength

isAccessAllowed



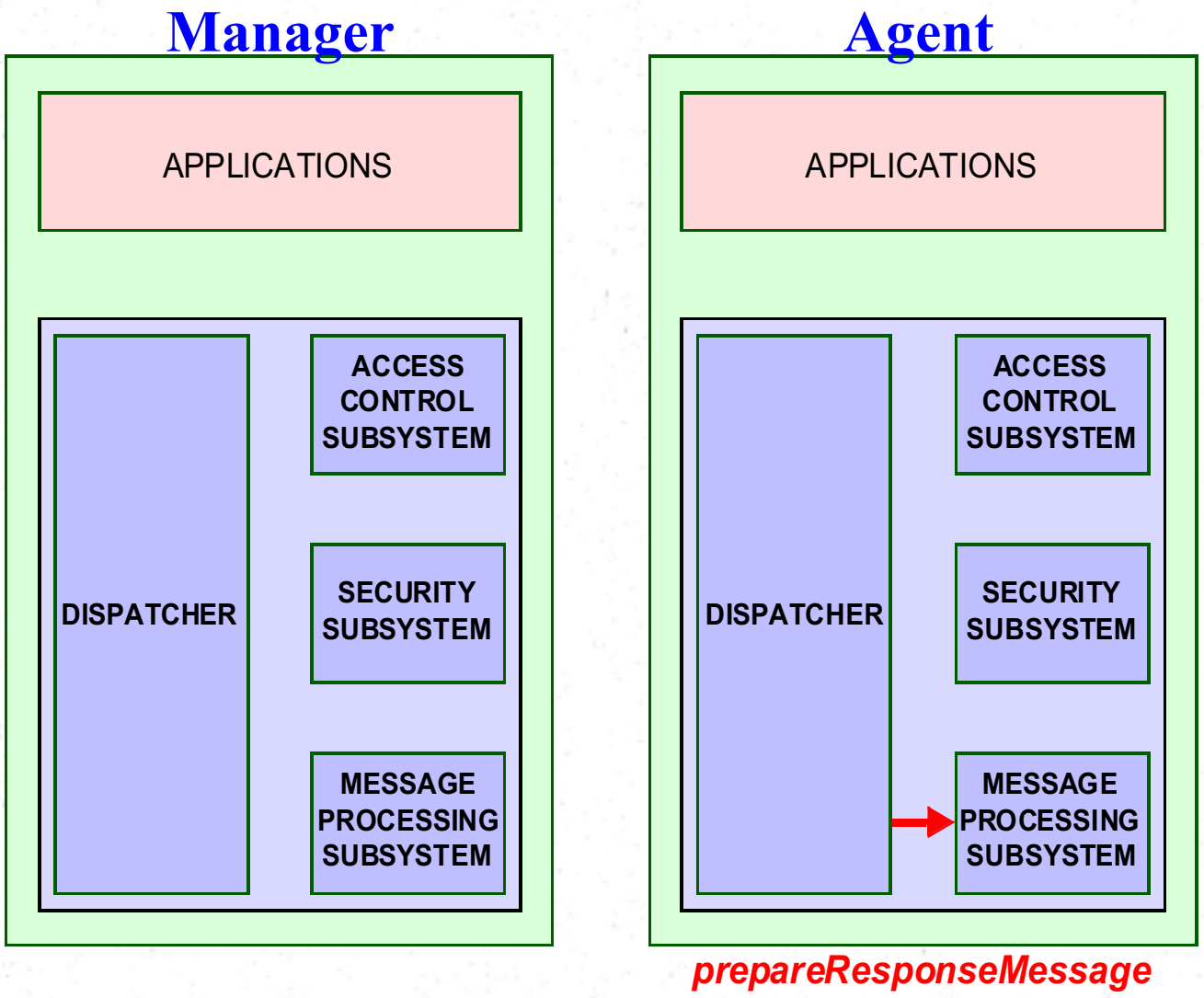
Parameters
contextEngineID
contextName
destTransportAddress
destTransportDomain
expectResponse
globalData
maxMessageSize
maxSizeResponseScopedPDU
messageProcessingModel
outgoingMessage
outgoingMessageLength
PDU
pduType
pduVersion
scopedPDU
stateReference
statusInformation
securityEngineID
securityLevel
securityModel
securityName
securityParameters
securityStateReference
sendPduHandle
transportAddress
transportDomain
variableName
viewType
wholeMsg
wholeMsgLength

returnResponsePdu



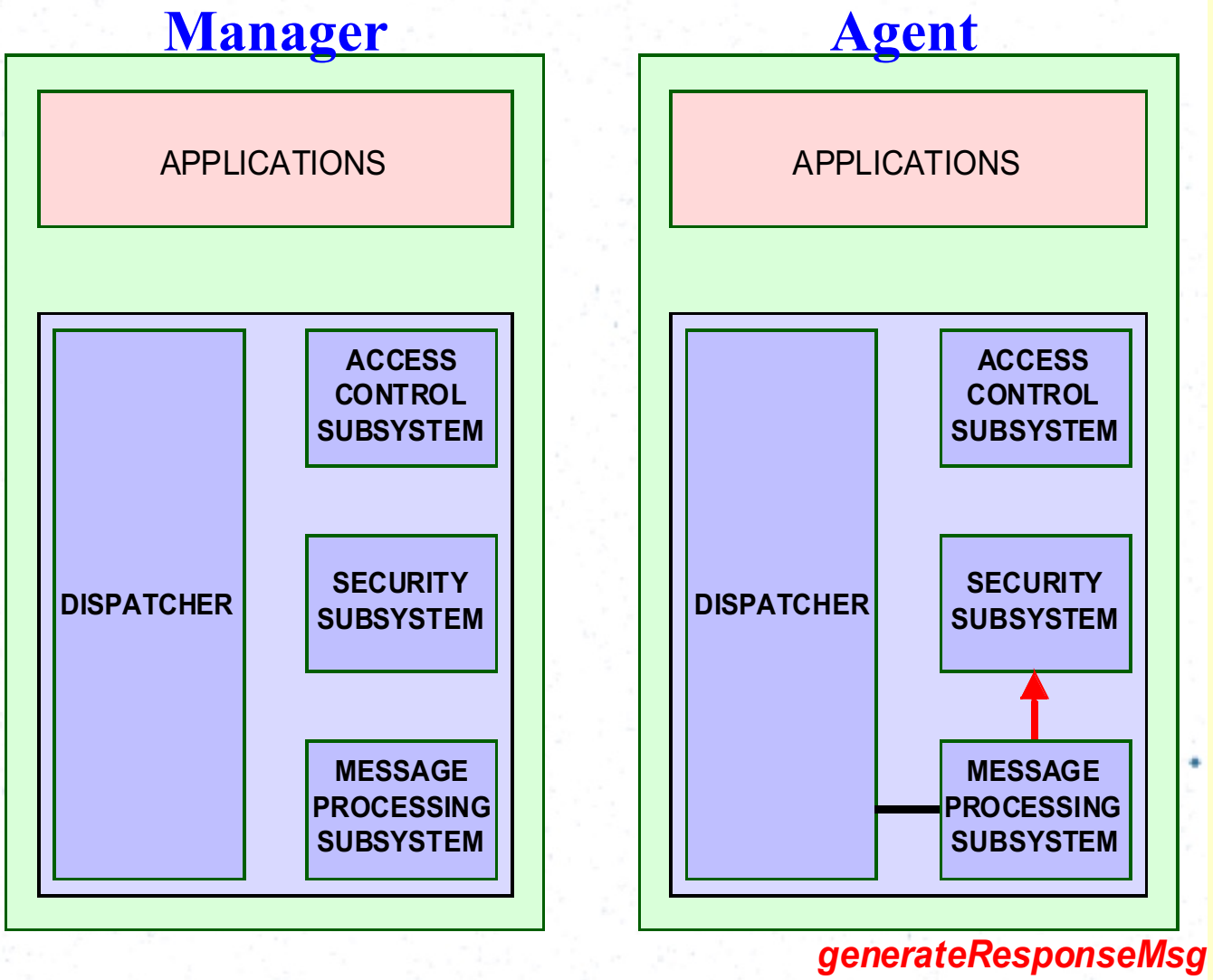
Parameters
contextEngineID
contextName
destTransportAddress
destTransportDomain
expectResponse
globalData
maxMessageSize
maxSizeResponseScopedPDU
messageProcessingModel
outgoingMessage
outgoingMessageLength
PDU
pduType
pduVersion
scopedPDU
stateReference
statusInformation
securityEngineID
securityLevel
securityModel
securityName
securityParameters
securityStateReference
sendPduHandle
transportAddress
transportDomain
variableName
viewType
wholeMsg
wholeMsgLength

prepareResponseMessage



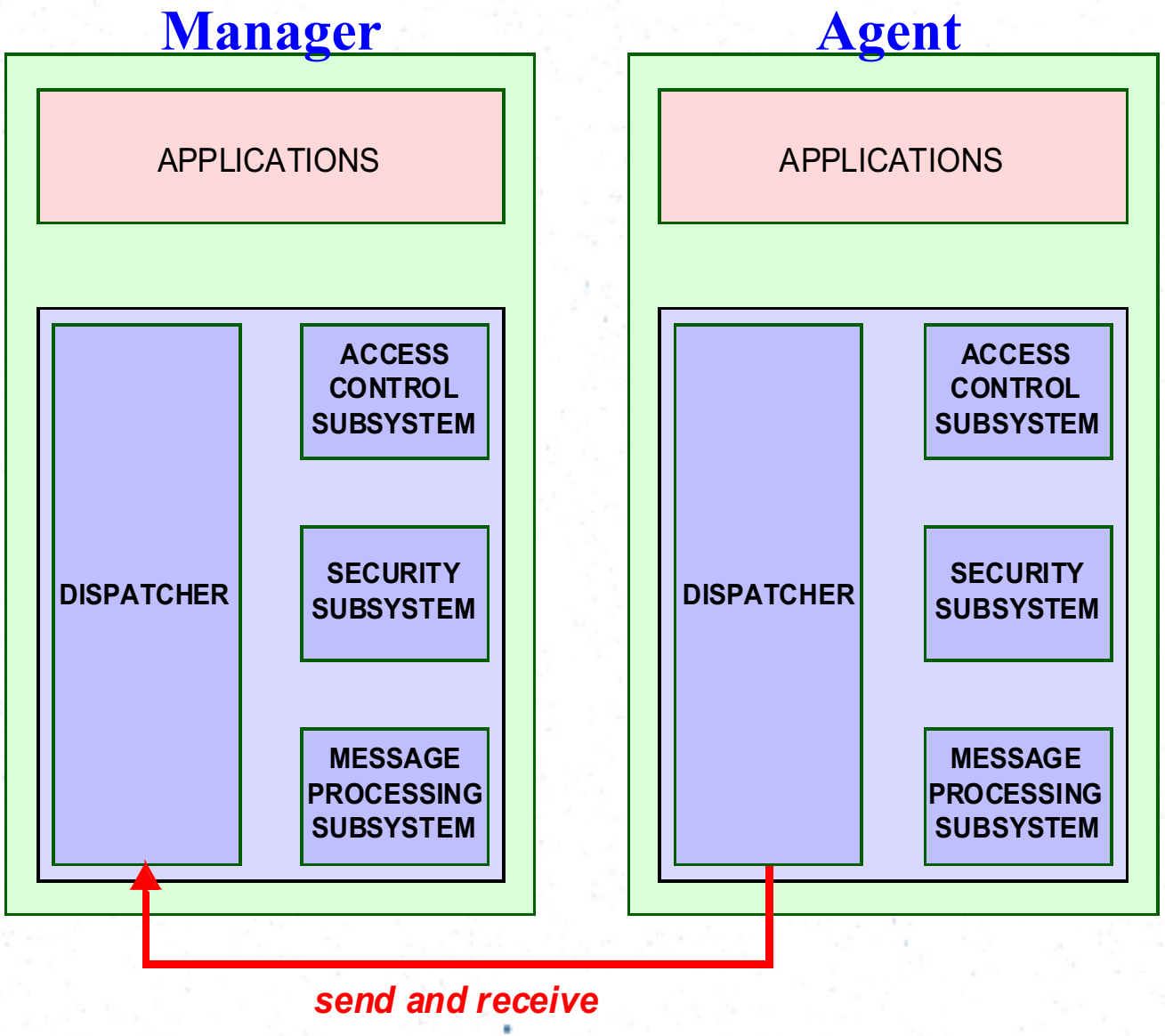
Parameters
contextEngineID
contextName
destTransportAddress
destTransportDomain
expectResponse
globalData
maxMessageSize
maxSizeResponseScopedPDU
messageProcessingModel
outgoingMessage
outgoingMessageLength
PDU
pduType
pduVersion
scopedPDU
stateReference
statusInformation
securityEngineID
securityLevel
securityModel
securityName
securityParameters
securityStateReference
sendPduHandle
transportAddress
transportDomain
variableName
viewType
wholeMsg
wholeMsgLength

generateResponseMsg



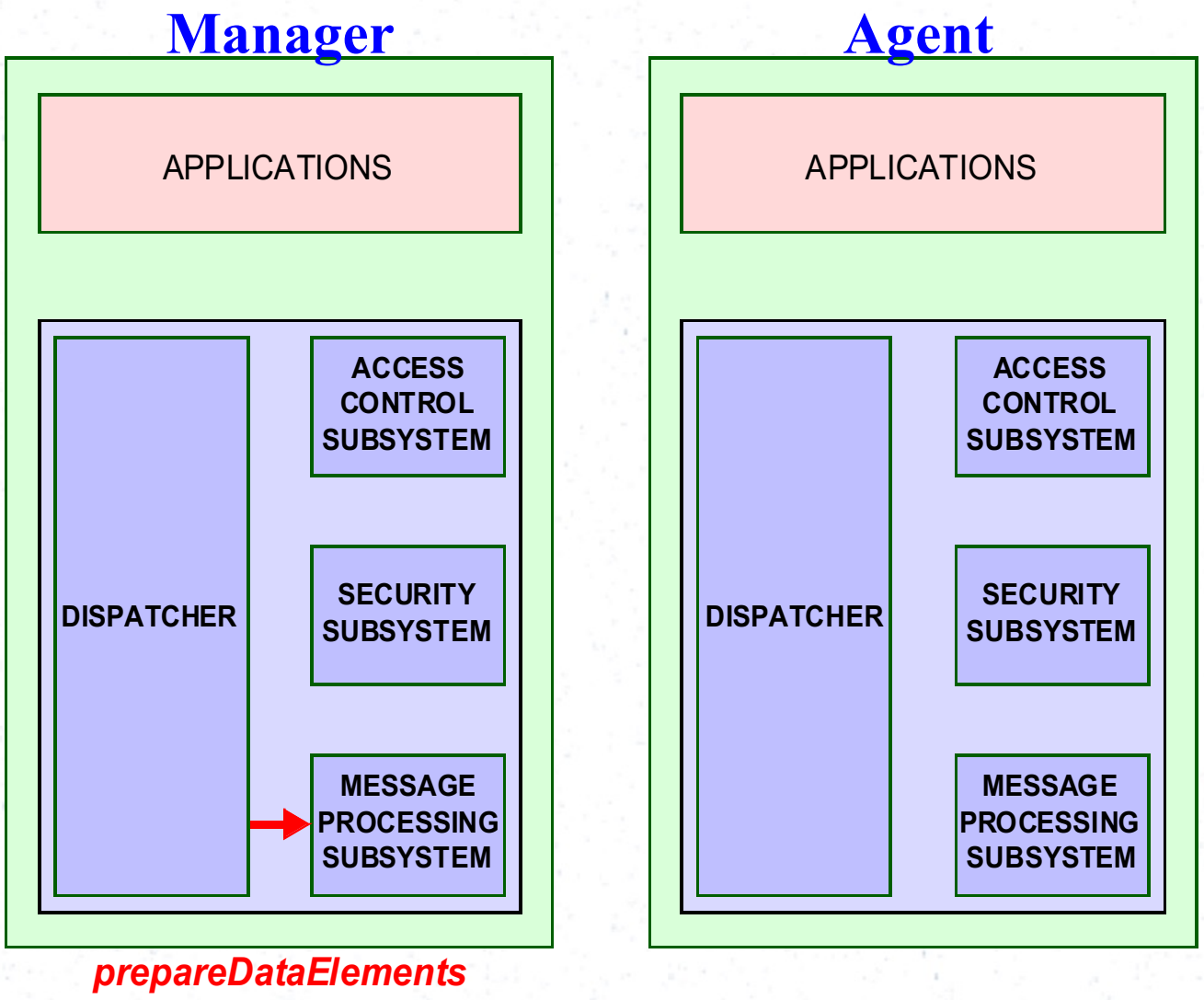
Parameters
contextEngineID
contextName
destTransportAddress
destTransportDomain
expectResponse
globalData
maxMessageSize
maxSizeResponseScopedPDU
messageProcessingModel
outgoingMessage
outgoingMessageLength
PDU
pduType
pduVersion
scopedPDU
stateReference
statusInformation
securityEngineID
securityLevel
securityModel
securityName
securityParameters
securityStateReference
sendPduHandle
transportAddress
transportDomain
variableName
viewType
wholeMsg
wholeMsgLength

send / receive



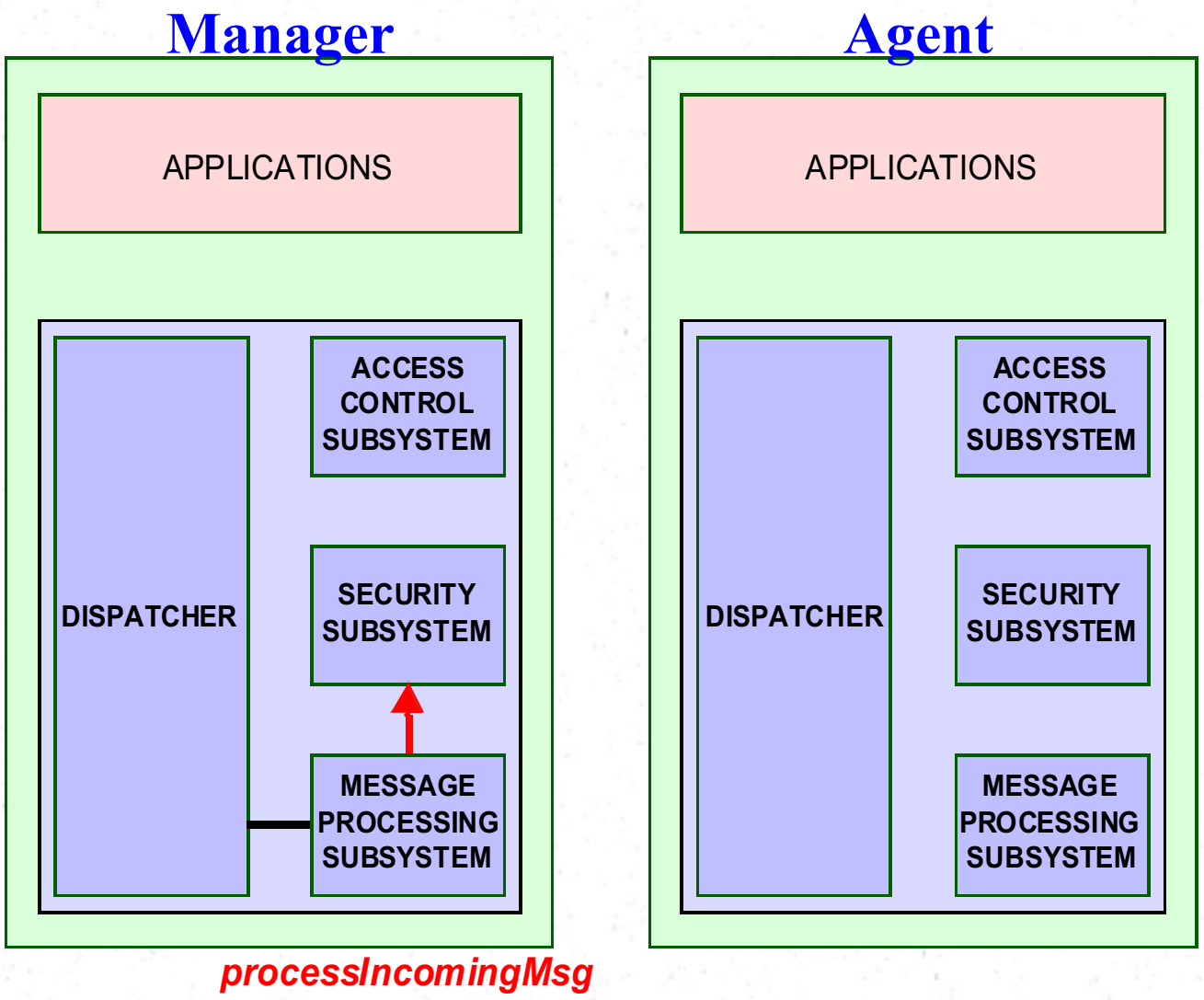
Parameters
contextEngineID
contextName
destTransportAddress
destTransportDomain
expectResponse
globalData
maxMessageSize
maxSizeResponseScopedPDU
messageProcessingModel
outgoingMessage
outgoingMessageLength
PDU
pduType
pduVersion
scopedPDU
stateReference
statusInformation
securityEngineID
securityLevel
securityModel
securityName
securityParameters
securityStateReference
sendPduHandle
transportAddress
transportDomain
variableName
viewType
wholeMsg
wholeMsgLength

prepareDataElements



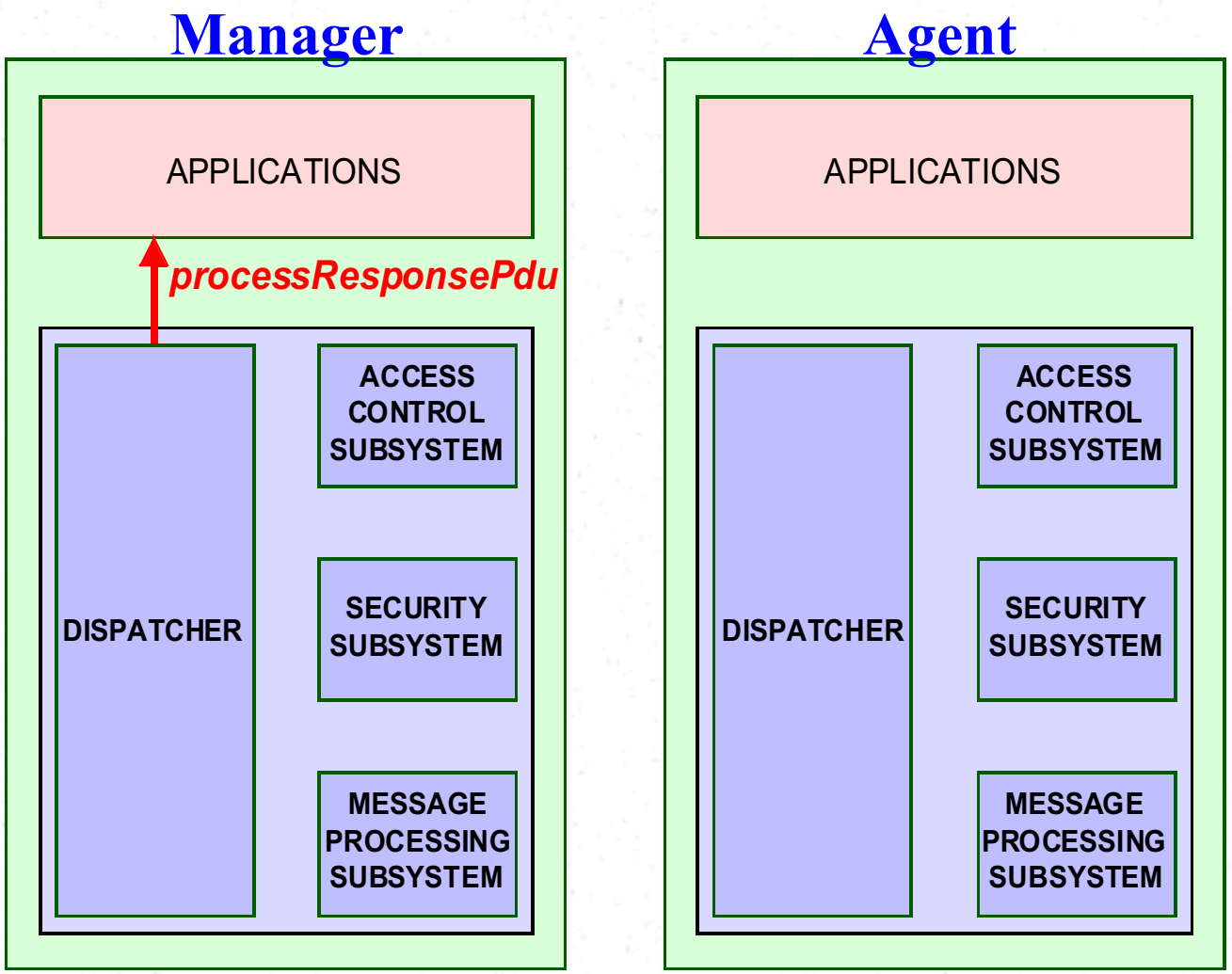
Parameters
contextEngineID
contextName
destTransportAddress
destTransportDomain
expectResponse
globalData
maxMessageSize
maxSizeResponseScopedPDU
messageProcessingModel
outgoingMessage
outgoingMessageLength
PDU
pduType
pduVersion
scopedPDU
stateReference
statusInformation
securityEngineID
securityLevel
securityModel
securityName
securityParameters
securityStateReference
sendPduHandle
transportAddress
transportDomain
variableName
viewType
wholeMsg
wholeMsgLength

processIncomingMsg



Parameters
contextEngineID
contextName
destTransportAddress
destTransportDomain
expectResponse
globalData
maxMessageSize
maxSizeResponseScopedPDU
messageProcessingModel
outgoingMessage
outgoingMessageLength
PDU
pduType
pduVersion
scopedPDU
stateReference
statusInformation
securityEngineID
securityLevel
securityModel
securityName
securityParameters
securityStateReference
sendPduHandle
transportAddress
transportDomain
variableName
viewType
wholeMsg
wholeMsgLength

processResponsePdu



Parameters
contextEngineID
contextName
destTransportAddress
destTransportDomain
expectResponse
globalData
maxMessageSize
maxSizeResponseScopedPDU
messageProcessingModel
outgoingMessage
outgoingMessageLength
PDU
pduType
pduVersion
scopedPDU
stateReference
statusInformation
securityEngineID
securityLevel
securityModel
securityName
securityParameters
securityStateReference
sendPduHandle
transportAddress
transportDomain
variableName
viewType
wholeMsg
wholeMsgLength

SNMP RFC's

RFC	Description	Published	Current Status
1065	SMIv1	Aug-88	Obsoleted by 1155
1066	SNMPv1 MIB	Aug-88	Obsoleted by 1156
1067	SNMPv1	Aug-88	Obsoleted by 1098
1098	SNMPv1	Apr-89	Obsoleted by 1157
1155	SMIv1	May-90	Standard
1156	SNMPv1 MIB	May-90	Historic
1157	SNMPv1	May-90	Standard
1158	SNMPv1 MIB-II	May-90	Obsoleted by 1213
1212	SNMPv1 MIB definitions	Mar-91	Standard
1213	SNMPv1 MIB-II	Mar-91	Standard
1215	SNMPv1 traps	Mar-91	Informational
1351	Secure SNMP administrative model	Jul-92	Proposed Standard
1352	Secure SNMP managed objects	Jul-92	Proposed Standard
1353	Secure SNMP security protocols	Jul-92	Proposed Standard
1441	Introduction to SNMPv2	Apr-93	Proposed Standard
1442	SMIv2	Apr-93	Obsoleted by 1902
1443	Textual conventions for SNMPv2	Apr-93	Obsoleted by 1903
1444	Conformance statements for SNMPv2	Apr-93	Obsoleted by 1904
1445	SNMPv2 administrative model	Apr-93	Historic

RFC	Description	Published	Current Status
1448	SNMPv2 protocol operations	Apr-93	Obsoleted by 1905
1449	SNMPv2 transport mapping	Apr-93	Obsoleted by 1906
1450	SNMPv2 MIB	Apr-93	Obsoleted by 1907
1451	Manger-to-manger MIB	Apr-93	Historic
1452	Coexistence of SNMPv1 and SNMPv2	Apr-93	Obsoleted by 1908
1901	Community-Based SNMPv2	Jan-96	Experimental
1902	SMIv2	Jan-96	Draft Standard
1903	Textual conventions for SNMPv2	Jan-96	Draft Standard
1904	Conformance statements for SNMPv2	Jan-96	Draft Standard
1905	Protocol operations for SNMPv2	Jan-96	Draft Standard
1906	Transport mapping for SNMPv2	Jan-96	Draft Standard
1907	SNMPv2 MIB	Jan-96	Draft Standard
1908	Coexistence of SNMPv1 and SNMPv2	Jan-96	Draft Standard
1909	Administrative infrastructure for SNMPv2	Feb-96	Experimental
1910	User-based security for SNMPv2	Feb-96	Experimental

SNMP Security

- SNMPv1 uses plain text community strings for authentication as plain text without encryption
- SNMPv2 was supposed to fix security problems, but effort derailed (The “c” in SNMPv2c stands for “community”).
-
- SNMPv3 has numerous security features:
  - Ensure that a packet has not been tampered with (**integrity**),
  - Ensures that a message is from a valid source (**authentication**)
  - Ensures that a message cannot be read by unauthorized (**privacy**).